

DP-LMI package

User Manual

Version v0.3.0

July 2026

Abstract

This manual describes the current public behavior of the DP-LMI package. The package uses `pdbase` as the backend parent for cell-local Bernstein coefficient storage, provides `pdmat` for known finite real matrix functions on tensor grids, provides `pdvar` for YALMIP-backed Bernstein decision expressions and `rhodiff` derivatives, and provides `pdlmi` for direct coefficient-wise assembly, opt-in Pólya elevation, and an opt-in cell-local full box Bernstein–Gram preordering. The manual is written as a reference manual: it begins with the canonical differentiable PDLMI and gridding model, then separates setup, public-class references, shared helpers, examples, and the mathematical Bernstein and positivity-certificate appendices.

Contents

1	Canonical Differentiable PDLMI	5
1.1	Problem Form	5
1.2	Gridding One, Two, And ℓ Parameters	5
1.3	Why Bernstein Coordinates	6
1.4	From Known Data To Solver Constraints	6
2	Installation, Verification, And Lookup	8
2.1	Installation	8
2.2	Verification	8
2.3	Smoke Test	8
2.4	Global Function Lookup	9
3	pdbase Backend Reference	10
3.1	Lookup	10
3.2	pdbase Constructor	10
3.3	Storage Fields	12
3.4	Inspection: <code>cells</code> , <code>lbls</code> , And <code>coeffs</code>	13
3.5	Counts And Shape	14
3.5.1	<code>elevVals</code>	15
3.5.2	<code>elevLocalValues</code>	16
3.5.3	<code>bernElev</code>	16
3.5.4	<code>bernProd</code>	17
3.5.5	<code>mergeGrid</code>	18
3.6	See Also	19
4	pdmat Reference	20
4.1	Lookup	20
4.2	Construction	20
4.3	<code>evaluate</code>	23
4.4	Visualization And Inspection	25
4.4.1	<code>bernsteinTable</code>	25
4.4.2	<code>disp</code> And <code>display</code>	26
4.4.3	<code>plot</code>	27
4.5	Algebra Operators	28
4.5.1	Operator-level entries	32
4.6	Matrix Methods	35

4.6.1	Shape Inspection And Identity	35
4.6.2	Transpose, Vectorization, Diagonals, And Reshape	36
4.6.3	Triangular Parts And Reductions	37
4.6.4	Reordering And Repetition	38
4.6.5	Block Assembly	38
4.6.6	Equality And MATLAB Indexing Protocol	39
4.7	Indexing And Assignment	40
4.8	Limits	41
4.8.1	See Also	41
5	pdvar Reference	42
5.1	Lookup	42
5.2	Construction	42
5.3	<code>rhodiff</code>	45
5.4	<code>bernsteinTable</code>	48
5.5	Affine And Mixed Algebra	49
5.5.1	Operator-level entries	51
5.6	Matrix Methods	53
5.6.1	Shape Inspection And Identity	53
5.6.2	Transpose, Vectorization, Diagonals, And Reshape	53
5.6.3	Triangular Parts And Reductions	55
5.6.4	Reordering And Repetition	56
5.6.5	Block Assembly	56
5.6.6	Equality And MATLAB Indexing Protocol	57
5.7	Indexing And Assignment	58
5.8	<code>value</code>	59
5.9	Comparisons To <code>pdlmi</code>	60
5.10	Limits	61
5.10.1	See Also	61
6	pdlmi Reference	62
6.1	Lookup	62
6.2	Construction And Comparisons	62
6.3	Direct Coefficient Assembly	64
6.4	Opt-In Pólya Assembly And <code>applyPolya</code>	65
6.5	Full Box Bernstein–Gram Preordering And <code>applyFullBoxPreorder</code>	66
6.6	<code>toYalmip</code>	67
6.7	See Also	68
6.8	Solver-Facing Use	69

7	Cross-Class Helper Reference	70
7.1	Shared Validation And Errors	71
7.2	Class-Private Helpers	71
8	Examples	72
8.1	Deterministic Full-Box Selection	72
8.2	Parameter-Dependent Lyapunov Solver Smoke	73
8.3	Rate-Dependent Block LMI Solver Smoke	74
A	Bernstein Basis Mathematics	76
A.1	Forward Coordinates And Tensor Basis	76
A.2	Exact Monomial Conversion	76
A.3	Lossless Degree Elevation	76
A.4	Products As Weighted Convolution	77
A.5	Differentiation	77
A.6	de Casteljau Evaluation And Subdivision	77
B	Polynomial Matrix Sign Certificates On A Hypercube	78
B.1	Full-Space SOS	78
B.2	Direct Bernstein Coefficients	78
B.3	Pólya-Type Coefficient Relaxation	78
B.4	Putinar Quadratic Modules	79
B.5	Schmüdgen And Full Preorderings	79
B.6	The One-Dimensional Markov–Lukács Forms	79
B.7	Relationships And Implementation Boundary	79
	References	81

1

Canonical Differentiable PDLMI

1.1 Problem Form

Let the scheduling vector be $\rho = (\rho_1, \dots, \rho_\ell) \in \mathcal{P}$, with rate $\dot{\rho} \in \mathcal{V}$, where both sets are boxes. A differentiable parameter-dependent LMI asks for matrix functions such as $P(\rho) = P(\rho)^\top$ and possibly additional scalar or matrix decisions such that, for every admissible $(\rho, \dot{\rho})$,

$$P(\rho) \succeq 0, \quad (1)$$

$$\mathcal{F}\left(\rho, \dot{\rho}, P(\rho), \frac{\partial P}{\partial \rho_1}, \dots, \frac{\partial P}{\partial \rho_\ell}\right) \preceq 0. \quad (2)$$

The residual $\mathcal{F}$ must be affine in the optimization variables after the known parameter-dependent matrices have been fixed. A common Lyapunov instance is

$$\sum_{s=1}^{\ell} \dot{\rho}_s \frac{\partial P}{\partial \rho_s} + P(\rho)A(\rho) + A(\rho)^\top P(\rho) \preceq 0.$$

The package does not discretize sample values and hope that the inequalities hold between them. It represents each cell residual exactly in a local Bernstein basis, enumerates the vertices of the rate box when $\dot{\rho}$ enters affinely, and then applies a finite sufficient matrix-sign certificate. Strict inequalities are modeled explicitly by the user, for example $P \succeq \varepsilon I$; the package inserts no hidden margin.

1.2 Gridding One, Two, And ℓ Parameters

For one scheduling parameter, choose strictly increasing nodes

$$\rho_{1,0} < \rho_{1,1} < \dots < \rho_{1,N_1-1}.$$

Cell $c_1 \in \{1, \dots, N_1 - 1\}$ is the interval $[\rho_{1,c_1-1}, \rho_{1,c_1}]$. Its forward local coordinate is

$$\alpha_1 = \frac{\rho_1 - \rho_{1,c_1-1}}{\rho_{1,c_1} - \rho_{1,c_1-1}} \in [0, 1].$$

Thus local label zero belongs to the lower endpoint and label m to the upper endpoint.

For two parameters, two node vectors form rectangular cells indexed by (c_1, c_2) . Each cell is mapped to $[0, 1]^2$ by the two forward coordinates

$$\alpha_s = \frac{\rho_s - \rho_{s,c_s-1}}{\rho_{s,c_s} - \rho_{s,c_s-1}}, \quad s = 1, 2.$$

The cell-local degree- m representation is

$$M_{c_1, c_2}(\alpha_1, \alpha_2) = \sum_{i_1=0}^m \sum_{i_2=0}^m C_{i_1, i_2}^{(c_1, c_2)} B_{i_1}^m(\alpha_1) B_{i_2}^m(\alpha_2).$$

For ℓ parameters, the Cartesian product of ℓ node vectors creates $\prod_{s=1}^{\ell} (N_s - 1)$ hyperrectangular cells. With $\mathbf{c} = (c_1, \dots, c_{\ell})$, $\boldsymbol{\alpha} \in [0, 1]^{\ell}$, and $\mathbf{i} = (i_1, \dots, i_{\ell})$,

$$M_{\mathbf{c}}(\boldsymbol{\alpha}) = \sum_{\mathbf{i} \in \{0, \dots, m\}^{\ell}} C_{\mathbf{i}}^{(\mathbf{c})} \prod_{s=1}^{\ell} B_{i_s}^m(\alpha_s), \quad B_i^m(\alpha) = \binom{m}{i} (1 - \alpha)^{m-i} \alpha^i.$$

There are $(m + 1)^{\ell}$ coefficients per cell. Physical-cell subscripts and local labels use the same Cartesian-product order, with earlier dimensions varying more slowly. This convention is visible through `cells`, `lbls`, and `coeffs`. Cell-local spline and gridding constructions for parameter-dependent LMIs provide the control-theoretic motivation for this partitioned architecture [3, 4].

1.3 Why Bernstein Coordinates

The monomial and Bernstein bases span the same polynomial space, but the Bernstein basis matches this package’s cell architecture.

- **Shared-face C^0 ownership.** Adjacent cells are continuous when the complete Bernstein coefficient arrays on their common face are the same. `pdvar` enforces this structurally by reusing the same YALMIP handles, including edges and corners in several dimensions; no continuity equality LMIs are needed.
- **Exact products by convolution.** Products of basis functions have binomial-scaled convolution coefficients. Matrix multiplication therefore stays in the same cell-local representation while preserving operand order.
- **Exact degree elevation.** Coefficients of different degrees can be aligned without changing the represented polynomial or adding decision freedom.
- **Finite sign evidence.** The basis is nonnegative and sums to one, so sign-definite coefficient matrices certify the matrix sign on the whole cell. Failure of this sufficient test is not proof of infeasibility.

Bernstein convex-hull bounds have long been used for interval-parameter robustness [2]; their relation to broader polynomial and LMI certificates is surveyed in [5].

Appendix A gives the conversion, elevation, product, differentiation, and subdivision formulas. Appendix B places direct coefficient tests and the implemented relaxations beside the standard SOS certificates.

1.4 From Known Data To Solver Constraints

The public modeling pipeline separates representation from certification:

1. Construct known matrix data with `pdmat(gridVectors, source)`. A source may be a function handle, a global Bernstein coefficient grid, or an explicit nested cell-local coefficient tree. Supplying `Degree=m` establishes coefficient evidence and validates the represented polynomial.
2. Construct affine decision functions with `pdvar`. Degree zero shares one decision matrix across the full grid; degree $m \geq 1$ creates a continuous global control lattice whose cell views share complete faces. `rhodiff` forms cell-local derivative rows and enumerates the requested rate-box vertices.

3. Assemble a residual using supported known-data and affine algebra. Comparisons `expr <= rhs` and `expr >= rhs` return a `pdlmi` object that retains the original residual and relation.
4. Choose the certificate. Direct coefficient-wise assembly is the default. `applyPolya` performs exact degree elevation followed by the same coefficient test. `applyFullBoxPreorder` instead builds independent cell- and rate-row Gram blocks and exact coefficient identities for the local box.
5. Call `toYalmip` and use ordinary YALMIP solver settings and `optimize`. Solver selection, diagnostics, and feasibility status belong to YALMIP, not to `pdlmi`.

2

Installation, Verification, And Lookup

2.1 Installation

Start MATLAB in the repository root, or set `projectRoot` to the absolute path of the checked-out repository. Add the package recursively, then remove the documentation directory from the executable path.

MATLAB input

```
projectRoot = "path/to/Differential Parameter-Dependent Linear Matrix Inequality";  
addpath(genpath(projectRoot));  
rmpath(genpath(fullfile(projectRoot, "doc")));
```

For local work from the repository root, this package also resolves from:

MATLAB input

```
addpath(pwd)
```

2.2 Verification

Run the current MATLAB test entry point from the repository root:

MATLAB input

```
results = tests.run_all();
```

The test entry point covers helper utilities, `pdbase`, `pdmat`, `pdvar`, and ordinary `pdlmi` behavior. The YALMIP-backed `pdvar` and `pdlmi` tests require YALMIP. Solver smoke tests additionally require an SDP solver. The optional cross-validation material in `sos_validation/`, including its full-box package oracle, is not loaded by `tests.run_all` and is not an ordinary package runtime dependency.

2.3 Smoke Test

First confirm that MATLAB resolves the four renamed constructors from this repository and not from another toolbox:

MATLAB input

```
which pdbase -all
which pdmat -all
which pdvar -all
which pdlmi -all
```

Then exercise known-data inspection and the solver-facing assembly boundary:

MATLAB input

```
A = pdmat({[0 1]}, {1, 3}, Degree=1);
T = bernsteinTable(A, "oneLine");
disp(T)
```

Output

CellSubscript	Expression
-----	-----
{[1]}	"(1-alpha)*1 + alpha*3"

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
F = toYalmip(C);
Cbox = C.applyFullBoxPreorder();
Fbox = toYalmip(Cbox);
```

F is the direct coefficient certificate; Fbox is the default full-box certificate for the same stored residual. Either can be passed to ordinary YALMIP calls, for example:

Call forms

```
sol = optimize(F, objective, opts);
```

2.4 Global Function Lookup

Category	Entries
Backend	pdbase , constructor , cells , lbls , coeffs , ncell , ncoeff , npar , size
Known data	pdmat , construction , evaluate , bernsteinTable , disp/display , plot , operators , matrix methods
Decision variables	pdvar , construction , rhodiff , affine/mixed algebra , matrix methods , comparisons
LMIs	pdlmi , construction , direct coefficient assembly , applyPolya , applyFullBoxPreorder , toYalmip , solver examples

3

pdbase Backend Reference

`pdbase` is the developer-facing backend parent for `pdmat` and `pdvar`. Ordinary users should model known data with `pdmat` and decision expressions with `pdvar`. This section documents `pdbase` because it explains the shared storage, traversal, and inspection contract used by both public modeling classes.

3.1 Lookup

Task	Function or field
Create backend object	<code>pdbase</code>
Inspect cells/labels	<code>cells</code> , <code>lbls</code> , <code>coeffs</code>
Count sizes	<code>ncell</code> , <code>ncoeff</code> , <code>npar</code> , <code>size</code>
Elevate stored coefficients	<code>elevVals</code>
Backend algebra mechanisms	<code>elevLocalValues</code> , <code>bernElev</code> , <code>bernProd</code> , <code>mergeGrid</code>
Read storage fields	<code>GridInfo</code> , <code>LocalValues</code> , <code>metadata fields</code>

3.2 pdbase Constructor

Syntax

Call forms

```
obj = pdbase(gridVectors, matrixSize, degree)
obj = pdbase(gridVectors, matrixSize, degree, localValues)
obj = pdbase(..., IsContinuous=tf)
obj = pdbase(..., ContainsDecision=tf)
obj = pdbase(..., HasRateDependence=tf)
obj = pdbase(..., RateBounds=rb)
obj = pdbase(..., SourceSummary=txt)
```

Arguments and options

- `gridVectors` is a cell vector with one strictly increasing node vector per parameter, such as `{[0 1 2], [10 20]}`.
- `matrixSize` is a positive two-element matrix size, such as `[2 2]`.

- **degree** is a nonnegative integer Bernstein degree, such as 1.
- **localValues** is the nested physical-cell storage. When omitted, **pdbase** creates a zero coefficient-backed object.
- **IsContinuous**, **ContainsDecision**, and **HasRateDependence** are logical metadata scalars. They default to false; subclasses set them to describe their own payload contract.
- **RateBounds** is empty or an $\ell \times 2$ matrix of finite lower and upper rate bounds, such as **RateBounds**=[-1 1; -2 2]. Nonempty bounds also mark the object as rate-dependent.
- **SourceSummary** is inspectable origin text and defaults to "coefficient-backed".

Example

This backend example constructs a two-parameter parent object. The object has two physical cells because the first parameter has two intervals and the second parameter has one interval.

MATLAB input

```
obj = pdbase({[0 1 2], [10 20]}, [2 2], 1);
class(obj)
obj.MatrixSize
obj.Degree
obj.ncell()
obj.ncoeff()
obj.npar()
size(obj)
```

Output

```
ans =
    'pdbase'

ans =
     2     2

ans =
     1

ans =
     2

ans =
     4

ans =
     2

ans =
     2     2
```

The printed **ncoeff** value is $(1 + 1)^2 = 4$, one local Bernstein coefficient for each tensor-product label in the two parameter directions.

Validation and errors

Malformed grid containers and node vectors raise `pdbase:InvalidGrid` and `pdbase:InvalidGridVector`. Invalid matrix sizes, degrees, or rate bounds raise `pdbase:InvalidMatrixSize`, `pdbase:InvalidDegree`, and `pdbase:InvalidRateBounds`. The nested tree must contain exactly one leaf per physical cell and $(m + 1)^\ell$ coefficient columns per leaf; violations raise `pdbase:InvalidLocalValues` or `pdbase:InvalidCoefficientCell`. Payload matrices must have the declared size and be finite, real, and either numeric or supported real YALMIP expressions. Malformed payloads raise `pdbase:InvalidCoefficientPayload`. Rate-affine payload structures and `HasRateDependence=true` require compatible nonempty `RateBounds`.

3.3 Storage Fields

Field	Meaning
<code>GridInfo</code>	Grid vectors, node counts, and grid bounds.
<code>MatrixSize</code>	Size of each matrix coefficient or matrix expression.
<code>Degree</code>	Local Bernstein degree stored in every physical cell.
<code>LocalValues</code>	Nested cell tree indexed by physical-cell subscript; each leaf stores coefficient cells.
<code>IsContinuous</code>	Metadata flag; subclasses are responsible for boundary sharing if continuity is true.
<code>ContainsDecision</code>	True when coefficient payloads include YALMIP decision variables.
<code>HasRateDependence</code>	True when the object carries rate metadata or rate-vertex rows.
<code>RateBounds</code>	Parameter-rate lower and upper bounds used by <code>rhodiff</code> and <code>pdlmi</code> .
<code>SourceSummary</code>	Short origin label such as <code>coefficient-backed</code> , <code>decision</code> , or <code>derivative</code> .

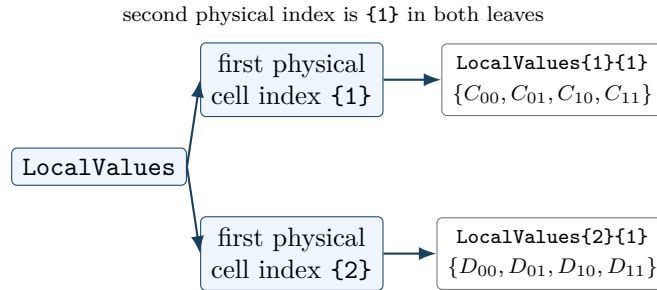


Figure 1: Nested cell-local storage for a two-parameter, degree-one object on $\{[0 \ 1 \ 2], [10 \ 20]\}$. There are two physical cells, $[1, 1]$ and $[2, 1]$. Each leaf is a flat local coefficient cell in `1b1s` order: $[0, 0]$, $[0, 1]$, $[1, 0]$, $[1, 1]$.

The nesting follows physical-cell subscripts, not the Bernstein labels. Start with `LocalValues\{i1\}`, then take one cell index per remaining parameter; this selects physical cell $[i_1, i_2, \dots, i_\ell]$. Only after that selection does the leaf's flat cell array enumerate the $(m + 1)^\ell$ local Bernstein coefficients. Thus `coeffs(obj, [2 1])` returns the lower leaf in fig. 1, while `1b1s(obj)` explains the order inside that leaf.

3.4 Inspection: `cells`, `lbls`, And `coeffs`

Syntax

Call forms

```
subs = cells(obj)
subs = obj.cells()

labels = lbls(obj)
labels = obj.lbls()

c = coeffs(obj, cellSubs)
c = obj.coeffs(cellSubs)
```

Arguments and options

- `obj` is a `pdbase` object or a `pdmat`/`pdvar` subclass instance using the shared cell-local storage convention.
- `cellSubs` selects one physical cell for `coeffs`. Supply a numeric row vector such as `[2 1]` or a cell array of scalar subscripts such as `{2,1}`.
- `cells` returns physical hypercube subscripts in the shared traversal order; `lbls` returns local Bernstein labels in the shared coefficient order.
- `c` is the flat coefficient cell array for the selected physical cell, ordered consistently with `lbls(obj)`.

Example

The label order is shared by `pdmat`, `pdvar`, and `pdlmi`. In this two-parameter degree-one object, each physical cell stores four coefficient entries with labels `[0,0]`, `[0,1]`, `[1,0]`, and `[1,1]`.

MATLAB input

```
obj = pdbase({[0 1 2], [10 20]}, [1 1], 1);
lbls(obj)
cells(obj)
c = coeffs(obj, [2 1]);
numel(c)
```

Output

```
ans =  
    0    0  
    0    1  
    1    0  
    1    1  
  
ans =  
    1    1  
    2    1  
  
ans =  
    4
```

3.5 Counts And Shape

Syntax

Call forms

```
n = ncell(obj)  
n = obj.ncell()  
  
n = ncoeff(obj)  
n = obj.ncoeff()  
  
n = npar(obj)  
n = obj.npar()  
  
sz = size(obj)  
d = size(obj, dim)
```

Arguments and options

- `obj` is a `pdbase` object or subclass instance.
- `dim` is a positive integer matrix dimension for `size`; it follows MATLAB's ordinary `size(obj,dim)` convention.
- `ncell` returns $\prod_r (N_r - 1)$, the number of physical hypercube cells.
- `ncoeff` returns $(m + 1)^\ell$, the number of local Bernstein coefficients stored per physical cell.
- `npar` returns the parameter dimension ℓ . `size` returns the matrix payload size, never the physical grid size.

Example

For a two-parameter degree-one backend object, the first grid has two physical intervals and the second has one. The matrix payload is 2-by-3, independently of the grid shape.

MATLAB input

```
obj = pdbase({[0 1 2], [10 20]}, [2 3], 1);  
fprintf('ncell = %d\n', obj.ncell());  
fprintf('ncoeff = %d\n', obj.ncoeff());  
fprintf('npar = %d\n', obj.npar());  
disp(size(obj))  
disp(size(obj, 1))  
disp(size(obj, 2))  
disp(size(obj, 3))
```

Output

```
ncell = 2  
ncoeff = 4  
npar = 2  
      2      3  
  
      2  
  
      3  
  
      1
```

3.5.1 elevVals

Syntax

Call forms

```
vals = obj.elevVals(degreeIncrement)
```

Arguments and options

- **degreeIncrement** is a finite nonnegative integer scalar. It is added to **obj.Degree** in every parameter direction; zero returns coefficient evidence at the current degree.

Output

`vals` is a nested `LocalValues`-shaped cell tree. Each physical cell contains coefficients of degree `obj.Degree + degreeIncrement`, with the represented polynomial, physical-cell tree, and any rate-row ordering preserved. The method does not modify `obj` or its stored coefficient evidence.

Validation and errors

An invalid increment raises `pdbase:InvalidDegreeIncrement`. A function-only `pdmatrix` has no stored Bernstein coefficient evidence, so this method raises `pdbase:MissingCoefficientEvidence`.

Relationship to backend and public algebra. Unlike protected `bernElev`, which elevates one flat cell coefficient family for internal alignment, `elevVals` is a public whole-object query returning elevated `LocalValues`. Public coefficient algebra uses `bernElev` internally when compatible operands need degree alignment; it does not mutate either operand.

3.5.2 `elevLocalValues`

Internal/protected mechanism. This method elevates a temporary nested coefficient tree after public algebra has remapped operands to a common physical grid. It is documented as backend context, not as a supported user call form.

Syntax

Call forms

```
vals = elevLocalValues(obj, vals, fromDeg, toDeg)
vals = elevLocalValues(obj, vals, fromDeg, toDeg, grid)
```

Each leaf must be a cell array with one column per degree-`fromDeg` Bernstein label. The method elevates each rate-vertex row independently to `toDeg`; it never mixes rows. The optional `grid` supplies the temporary physical-cell layout, and otherwise defaults to `obj.GridInfo.Vectors`. Equal source and target degrees return the input tree unchanged. A malformed leaf raises `pdbase:InvalidCoefficientCell`. Public users encounter this behavior through grid merging, assignment, concatenation, and arithmetic rather than by calling the protected method directly.

3.5.3 `bernElev`

Internal/protected mechanism. This signature documents the backend algorithm used by public algebra; it is not a supported user call form.

Syntax

Call forms

```
out = bernElev(obj, coeffs, fromDeg, toDeg)
```

Arguments and options

- `coeffs` is a flat coefficient-cell family in the same label order as `lbls(obj)`.
- `fromDeg` and `toDeg` are nonnegative integer degrees, with `toDeg` $\geq$ `fromDeg`.
- `out` contains $(\text{toDeg} + 1)^\ell$ elevated coefficients in the same label order.

Example

MATLAB input

```
A = pdmat({[0 1]}, {[1 2]}, Degree=1);  
B = pdmat({[0 1]}, {1, 2, 3}, Degree=2);  
C = A + B;  
C.Degree
```

Output

```
ans =  
    2
```

The example invokes the protected helper through public addition; users do not call `bernElev` directly.

3.5.4 bernProd

Internal/protected mechanism. This signature documents the backend algorithm used by public matrix multiplication; it is not a supported user call form.

Syntax

Call forms

```
out = bernProd(obj, lhs, lhsDeg, rhs, rhsDeg)
```

Arguments and options

- `lhs` and `rhs` are flat coefficient-cell families whose matrix payload inner dimensions are compatible for multiplication.
- `lhsDeg` and `rhsDeg` are their nonnegative local Bernstein degrees.
- `out` is the binomial-normalized coefficient convolution at degree `lhsDeg + rhsDeg`; payload products follow MATLAB matrix multiplication rules.

Example

MATLAB input

```
A = pdmat({[0 1]}, {[1 2]}, Degree=1);  
P = A * A;  
P.Degree
```

Output

```
ans =  
    2
```

The example invokes the protected helper through public matrix multiplication; users do not call `bernProd` directly.

3.5.5 `mergeGrid`

Internal/protected mechanism. This signature documents the backend grid-refinement algorithm used by public algebra; it is not a supported user call form.

Syntax

Call forms

```
grid = obj.mergeGrid(errId, other1, other2, ...)
```

Arguments and options

- `errId` is the error identifier used when parameter counts or physical bounds are incompatible.
- `other1`, `other2`, ... are coefficient-backed operands, including function-backed operands constructed with an explicit `Degree`. They must have equal parameter counts and matching first and last nodes on every parameter axis.

- `grid` has the sorted union of the interior nodes. Public algebra re-expresses operands on these cells before degree elevation, coefficient-wise addition, or product convolution.

This protected helper is invoked by public algebra methods rather than called directly by users.

Example

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);  
B = pdmat({[0 0.5 1]}, {10, 20, 30}, Degree=1);  
C = A + B;  
C.GridInfo.Vectors{1}
```

Output

```
ans =  
      0      0.5000      1.0000
```

The example invokes the protected helper through public addition; users do not call `mergeGrid` directly.

3.6 See Also

`pdmat`, `pdvar`, `cells`, `coeffs`, `lbls`, `ncell`, `ncoeff`, `npar`, `size`, `elevVals`, `elevLocalValues`, `bernElev`, `bernProd`, `mergeGrid`.

4

pdmat Reference

`pdmat` stores known finite real matrix data on a tensor grid. Objects can be coefficient-backed, function-only, or function-backed with explicit Bernstein coefficient evidence. Coefficient-backed objects participate in algebra. Function-only objects evaluate exactly through the stored handle, but do not expose coefficient evidence for `bernsteinTable` or coefficient algebra.

4.1 Lookup

Task	Entry
Construct data	<code>pdmat</code>
Evaluate	<code>evaluate</code>
Inspect/display	<code>bernsteinTable</code> , <code>disp</code> , <code>display</code>
Plot	<code>plot</code>
Arithmetic	<code>+</code> , <code>-</code> , unary <code>+/-</code> , <code>*</code>
Matrix/index methods	shape and structural methods, indexing and assignment

4.2 Construction

Syntax

Call forms

```
A = pdmat(gridVector, source)
A = pdmat(gridVector, source, Degree=m)
A = pdmat(gridVectors, source)
A = pdmat(gridVectors, source, Degree=m)
```

Arguments and options

- `gridVector` is a numeric vector used as a one-parameter grid shorthand, such as `[0 1 2]`.
- `gridVectors` is a cell vector with one numeric grid vector per parameter, such as `{[0 1], [10 20]}`.
- `source` may be a function handle, a global cell grid of numeric Bernstein coefficients, or explicit nested `LocalValues`; for example, `{1, 2, 3}` is scalar degree-two data on one cell.

- **Degree=m** sets the local Bernstein degree. For function handles, omitting **Degree** creates a function-only object; passing a value such as **Degree=2** validates and stores Bernstein coefficient evidence while retaining the exact function handle for **evaluate**.

Example

The scalar-grid shorthand is the shortest coefficient-backed form. The three numeric cells below are the global Bernstein coefficients for two physical cells with degree one.

MATLAB input

```
A = pdmat([0 1 2], {10, 20, 30}, Degree=1)
size(A)
A.Degree
A.SourceSummary
```

Output

```
A =
  pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     1

ans =
     1

ans =
  "coefficient-backed"
```

For tensor grids, pass one grid vector per parameter. The source cell array matches the global coefficient grid implied by the two parameter directions.

MATLAB input

```
B = pdmat({[0 1], [10 20]}, {1 2; 3 4}, Degree=1)
B.GridInfo.Vectors{2}
B.lbls()
```

Output

```
B =
  pdmat [1 1] over 2-D grid, degree 1, source coefficient-backed

ans =
    10    20

ans =
     0     0
     0     1
     1     0
     1     1
```

Use explicit nested **LocalValues** when each physical cell must be written directly. Here each cell stores two 1×2 row-vector coefficients.

MATLAB input

```
localVals = {[[1 0], [0 1]], {[2 0], [0 2]}};  
C = pdmat({[0 1 2]}, localVals, Degree=1)  
size(C)  
C.coeffs(2)
```

Output

```
C =  
  pdmat [1 2] over 1-D grid, degree 1, source coefficient-backed  
  
ans =  
      1      2  
  
ans =  
  1x2 cell array  
    {[2 0]}    {[0 2]}
```

When **source** is a function handle and **Degree** is omitted, the object keeps the exact evaluator but does not claim coefficient evidence.

MATLAB input

```
F = pdmat({[0 1]}, @(rho) [rho rho^2])  
F.SourceSummary  
F.evaluate(0.5)
```

Output

```
F =  
  pdmat [1 2] over 1-D grid, degree 1, source function  
  
ans =  
  "function"  
  
ans =  
    0.5000    0.2500
```

Add **Degree** for a polynomial function when later coefficient inspection or algebra should use Bernstein evidence. The exact function handle is still used for point evaluation.

MATLAB input

```
G = pdmat({[0 1]}, @(rho) rho.^2, Degree=2)  
G.SourceSummary  
G.coeffs(1)
```

Output

```
G =  
  pdmat [1 1] over 1-D grid, degree 2, source function-bernstein  
  
ans =  
  "function-bernstein"  
  
ans =  
  1x3 cell array  
    {[0]}    {[0]}    {[1]}
```

These forms all create finite known-data objects, but only the coefficient- backed and function-Bernstein forms can participate in coefficient algebra.

Public property

`FunctionHandle` retains the supplied function handle for function-only and function-Bernstein sources; it is empty for cell-backed sources. Its set access is private.

Validation and errors

Constructor options must be name–value pairs. Malformed syntax raises `pdmatrix:InvalidOptions`; backend-only options raise `pdmatrix:UnsupportedOption`; unknown names raise `pdmatrix:UnknownOption`. Grid errors use `pdmatrix:InvalidGrid` or `pdmatrix:InvalidGridVector`.

An unsupported source kind raises `pdmatrix:InvalidSource`. Function arity or initial-call failures raise `pdmatrix:InvalidFunctionHandle`; empty, complex, or otherwise invalid initial outputs raise `pdmatrix:InvalidFunctionOutput`. Global coefficient grids with inconsistent finite-real payload sizes raise `pdmatrix:InvalidData`. Incompatible inferred or explicit degrees raise `pdmatrix:InvalidDegree`; malformed nested cell storage raises `pdmatrix:InvalidLocalValues`. Malformed explicit coefficient-leaf counts or shapes raise `pdmatrix:InvalidCoefficientCell`; invalid, nonfinite, nonreal, or wrong-size coefficient payloads raise `pdmatrix:InvalidCoefficientPayload`. With an explicit `Degree`, a function that is not representable at that degree raises `pdmatrix:NonBernsteinPolynomial`; a failed or size-inconsistent validation probe raises `pdmatrix:InvalidFunctionValue`. Nested local data with mismatched shared faces is retained as discontinuous data and emits warning `pdmatrix:DiscontinuousLocalValues`.

4.3 evaluate

Syntax

Call forms

```
val = evaluate(A, point)
val = A.evaluate(point)
```

Arguments and options

- `A` is a `pdmatrix` object.
- `point` is a finite real scalar for a one-parameter object, such as `0.25`, or a finite real row vector with one entry per parameter, such as `[0.25 12]`. Every entry must lie within the corresponding grid bounds.

- `val` is a numeric matrix with `size(A)`. Coefficient-backed objects use local Bernstein interpolation; function-backed objects call their stored function handle.

Validation and errors

A point with the wrong type, length, or finite-real shape raises `pdmat:InvalidPoint`; a point outside any grid bound raises `pdmat:PointOutOfBounds`. If a stored function fails or returns a nonfinite, complex, empty, or size-inconsistent matrix at the requested point, `evaluate` raises `pdmat:InvalidFunctionValue`.

For a degree-one scalar coefficient-backed object on $[\rho_0, \rho_1]$,

$$A(\rho) = (1 - \alpha)A_0 + \alpha A_1, \quad \alpha = \frac{\rho - \rho_0}{\rho_1 - \rho_0}.$$

The same local-coordinate rule is applied entrywise for matrix-valued data.

Example

Use the function-call form when the object is naturally one input among several MATLAB expressions.

MATLAB input

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = evaluate(A, 0.25)
```

Output

```
val =
    2.5000
```

At $\rho = 0.25$, the public local coordinate is $\alpha = 0.25$, so the interpolated value is $0.75 \cdot 2 + 0.25 \cdot 4 = 2.5$. The dot-call form is equivalent and can be convenient in command-window exploration.

MATLAB input

```
A = pdmat({[0 1]}, {2, 4}, Degree=1);
val = A.evaluate(0.75)
```

Output

```
val =
    3.5000
```

Function-backed objects route through the retained function handle, so non-polynomial expressions can still be evaluated exactly at a point.

MATLAB input

```
F = pdmat({[0 pi]}, @(rho) sin(rho));  
val = F.evaluate(pi/2)
```

Output

```
val =  
    1
```

4.4 Visualization And Inspection

4.4.1 bernsteinTable

Syntax

Call forms

```
T = bernsteinTable(A)  
T = bernsteinTable(A, cellSubs)  
T = bernsteinTable(A, "oneLine")  
T = bernsteinTable(A, cellSubs, "oneLine")
```

Arguments and options

- **cellSubs** selects one physical cell.
- **"oneLine"** returns one row per selected physical cell with a compact Bernstein expression.
- Without **"oneLine"**, the full table uses seven metadata columns; the output below shows their exact MATLAB names.

For one parameter, the **Basis** column reports

$$\binom{m}{j} (1 - \alpha)^{m-j} \alpha^j.$$

For multiple parameters, the basis is the tensor product of that expression in each local coordinate, using **alpha1**, **alpha2**, and so on. **LocalIndex** is the local Bernstein label. **CoeffSubscript** maps the local label back to the global coefficient subscript on the physical grid. **IsPhysicalNode** is true when that coefficient lies on a physical grid node.

Example

MATLAB input

```
A = pdmat({[0 1]}, {[0 1], [1 2]}, Degree=1);  
T = bernsteinTable(A, 1, "oneLine");  
disp(T)
```

Output

CellSubscript	Expression
-----	-----
{[1]}	"(1-alpha)*[0 1] + alpha*[1 2]"

Use the full table when the metadata columns are the point of the example:

MATLAB input

```
A = pdmat({[0 1]}, {1, 2, 3}, Degree=2);  
T = bernsteinTable(A);  
disp(T)
```

Output

TermIndex	CellSubscript	CoeffSubscript	LocalIndex	Basis	IsPhysicalNode	Value
-----	-----	-----	-----	-----	-----	-----
1	{[1]}	{[1]}	{[0]}	"(1-alpha)^2"	true	{[1]}
2	{[1]}	{[2]}	{[1]}	"2(1-alpha)alpha"	false	{[2]}
3	{[1]}	{[3]}	{[2]}	"alpha^2"	true	{[3]}

Validation and errors

A function-only object has no coefficient evidence and raises `pdmat:FunctionOnlyBernsteinTable`. Unknown text options, repeated selectors, or invalid physical-cell selectors raise `pdmat:InvalidBernsteinTableInput`.

4.4.2 disp And display

Syntax

Call forms

```
disp(A)  
display(A)  
A
```

Arguments and options

`disp(A)` prints a compact one-line summary. `display(A)` is the command-window display used when an expression is not terminated by a semicolon; it prints grid, degree, coefficient, and source metadata.

Example

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);  
disp(A)  
display(A)
```

Output

```
pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed  
A =  
pdmat with 1-by-1 matrix values  
Parameters: 1  
  rho_1: [0, 1], 2 nodes  
Degree: 1  
Physical cells: 1  
Coefficients per cell: 2  
Continuous: true  
Source: coefficient-backed  
Function handle: false
```

4.4.3 plot

Syntax

Call forms

```
h = plot(A)  
h = plot(A, dims)  
h = plot(A, SamplesPerCell=n)  
h = plot(A, dims, SamplesPerCell=n)  
h = plot(A, ..., Name, Value)
```

Arguments and options

- `dims` is one or two parameter indices. For one-parameter objects, the default is 1; otherwise the default is `[1 2]`, as in `plot(A, [1 2])`.

- `SamplesPerCell` is the package-owned sampling option. The default is 15; use `SamplesPerCell=4` for a coarse diagnostic plot.
- Remaining name-value pairs pass through to MATLAB graphics. One- dimensional plots pass them to `plot`; two-dimensional plots pass them to `surf`. Examples include `LineWidth`, `FaceAlpha`, and `EdgeColor`.
- The output `h` is the vector of graphics handles, one entry per matrix element.

Validation and errors

An invalid parameter index, a repeated dimension, or a request for more than two plotting dimensions raises `pdmatrix:InvalidPlotDimensions`. Malformed plotting options or a nonpositive integer `SamplesPerCell` raise `pdmatrix:InvalidPlotOptions`. Errors from MATLAB graphics options remain MATLAB graphics errors.

Example

The one-dimensional call returns one graphics handle per matrix entry. The example inspects the generated sample count and line styling without relying on an interactive figure window.

MATLAB input

```
fig = figure('Visible','off');
A = pdmat({[0 1]}, @(rho) [rho, rho^2]);
h = plot(A, SamplesPerCell=4, LineWidth=2);
numel(h)
numel(h(1).XData)
h(1).LineWidth
h(1).YData([1 end])
close(fig)
```

Output

```
ans =
    2

ans =
    5

ans =
    2

ans =
    0    1
```

With four samples per cell on one physical interval, MATLAB stores five plotted node values including both endpoints.

4.5 Algebra Operators

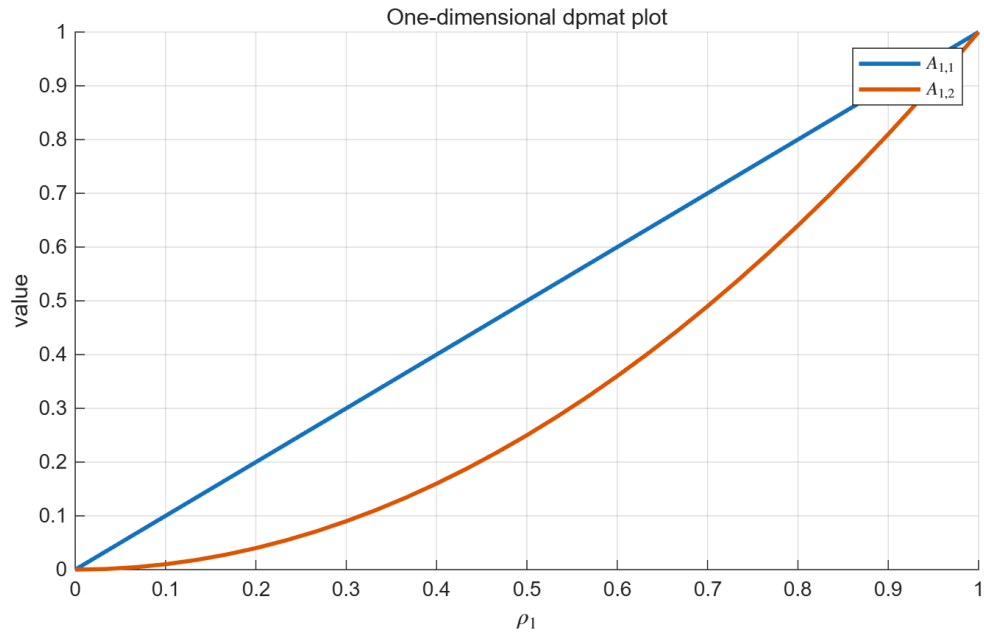


Figure 2: One-dimensional `pdmat/plot` output generated from MATLAB.

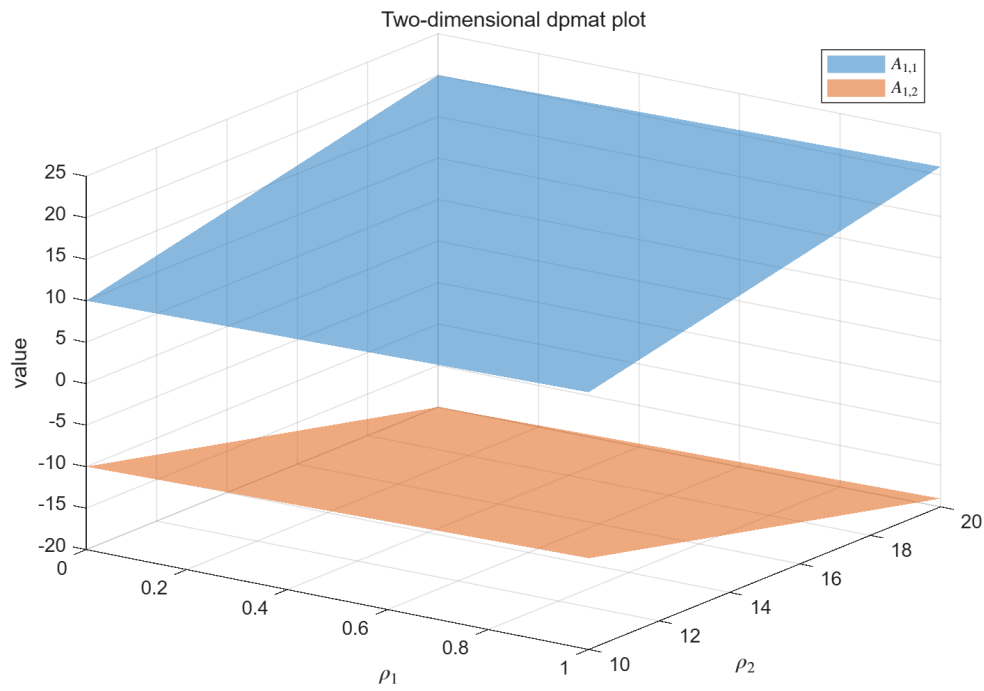


Figure 3: Two-dimensional `pdmat/plot` surface output generated from MATLAB.

Syntax

Call forms

```
C = A + B
C = A + M
C = M + A
C = A - B
C = A - M
C = M - A
C = +A
C = -A
C = A * B
C = A * M
C = M * A
```

Arguments and options

- Coefficient-backed **pdmat**: addition, subtraction, unary signs, and multiplication use local Bernstein coefficients.
- Numeric scalar, such as **5**, broadcasts to the **pdmat** matrix size when needed.
- Numeric matrix: accepted when the matrix size is compatible with the operation, such as **eye(2)** for a 2×2 object.
- Mixed grids with same bounds: re-expressed on a common refinement.
- Mixed degrees: elevated to a common degree for coefficient-wise operations; multiplication produces the degree sum.
- Function-only **pdmat**: evaluates and plots, but does not enter coefficient algebra unless explicit **Degree** supplied coefficient evidence.

For addition and subtraction, the package elevates both operands to a common degree before combining matching coefficients:

$$C_j = \hat{A}_j + \hat{B}_j.$$

For multiplication, coefficient labels convolve and the output degree is the sum of the input degrees.

Example

Addition elevates the lower-degree operand before combining coefficients.

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
S = A + B
cS = S.coeffs(1);
[cS{:}]
```

Output

```
S =
  pdmat [1 1] over 1-D grid, degree 2, source coefficient-backed

ans =
    11.0000    21.5000    32.0000
```

Operands with the same bounds but different node densities are independently re-expressed on the union grid before their coefficients are combined.

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 0.5 1]}, {10, 20, 30}, Degree=1);
S = A + B;
c1 = S.coeffs(1);
c2 = S.coeffs(2);
fprintf('Merged grid:\n');
disp(S.GridInfo.Vectors{1})
fprintf('Degree: %d\n', S.Degree);
fprintf('Cell 1 coefficients:\n');
disp([c1{:}])
fprintf('Cell 2 coefficients:\n');
disp([c2{:}])
```

Output

```
Merged grid:
     0    0.5000    1.0000

Degree: 1
Cell 1 coefficients:
    11.0000    21.5000

Cell 2 coefficients:
    21.5000    32.0000
```

Multiplication raises degree from 1 and 2 to 3.

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
B = pdmat({[0 1]}, {10, 20, 30}, Degree=2);
P = A * B
cP = P.coeffs(1);
[cP{:}]
```

Output

```
P =
  pdmat [1 1] over 1-D grid, degree 3, source coefficient-backed

ans =
    10.0000    20.0000    36.6667    60.0000
```

Numeric scalars are promoted to compatible constant coefficient data.

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
M = 5 - A
N = 2 * A
cM = M.coeffs(1);
cN = N.coeffs(1);
[cM{:}]
[cN{:}]
```

Output

```
M =
  pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed

N =
  pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     4     3

ans =
     2     4
```

These scalar-cell examples expose degree elevation, refinement, and numeric promotion separately. Tensor multiplication is shown under `mtimes` below.

4.5.1 Operator-level entries

plus. Adds two coefficient-backed objects or adds a compatible numeric scalar/matrix. The supported forms are $C=A+B$, $C=A+M$, and $C=M+A$; grids are refined and degrees are elevated before coefficient-wise addition. For example, `size(A+3)` returns the same matrix size as `A`.

Syntax

Call forms

```
C = A + B
C = A + M
C = M + A
```

minus. Subtracts compatible known-data objects or numeric operands. $A-M$ and $M-A$ preserve the left/right order of the matrix subtraction.

Syntax

Call forms

```
C = A - B  
C = A - M  
C = M - A
```

uplus and **uminus**. Unary plus returns an equivalent coefficient-backed object; unary minus negates every local coefficient.

Syntax

Call forms

```
B = +A  
B = -A
```

mtimes. Performs a matrix product, scalar product, or numeric matrix product. The inner matrix dimensions must agree and the output degree is the sum of operand degrees for two parameter-dependent operands.

Syntax

Call forms

```
C = A * B  
C = A * M  
C = M * A
```

Example

For a two-parameter cell, multiplying two degree-one scalar value grids raises **Degree** to 2 and produces $(2 + 1)^2 = 9$ tensor coefficients.

MATLAB input

```
grid = {[0 1], [10 20]};  
A = pdmat(grid, {1 2; 3 4}, Degree=1);  
B = pdmat(grid, {5 6; 7 8}, Degree=1);  
K = A * B;  
cK = K.coeffs([1 1]);  
fprintf('Tensor product degree: %d\n', K.Degree);  
fprintf('Tensor labels:\n');  
disp(K.lbls())
```

```
fprintf('Tensor coefficients:\n');
disp([cK{:}])
```

Output

```
Tensor product degree: 2
Tensor labels:
    0    0
    0    1
    0    2
    1    0
    1    1
    1    2
    2    0
    2    1
    2    2

Tensor coefficients:
    5     8    12    11    15    20    21    26    32
```

Example

Unary signs and numeric addition preserve the implemented `pdmat` type and evaluation behavior.

MATLAB input

```
A = pdmat({[0 1]}, {1, 2}, Degree=1);
class(+A)
(-A).evaluate(0)
(A + 3).evaluate(1)
```

Output

```
ans =
    'pdmat'

ans =
    -1

ans =
     5
```

Validation and errors

Incompatible operands for `+`, `-`, and `*` raise `pdmatrix.InvalidAddition`, `pdmatrix.InvalidSubtraction`, or `pdmatrix.InvalidMultiplication`. Parameter counts or outer bounds that cannot be merged raise `pdmatrix.MixedGrid`. Coefficient algebra on a function-only object raises `pdmatrix.FunctionOnlyAlgebra`. Exact equality comparisons that cannot form a compatible comparison grid raise `pdmatrix.InvalidEquality`.

4.6 Matrix Methods

Matrix methods act on every local Bernstein matrix coefficient; they do not rearrange the scheduling grid. Structural operations require coefficient evidence. A function-only `pdmatrix` can still use `evaluate`, `plot`, `uplus`, `squeeze`, shape inspection, and equality; other structural operations reject it with `pdmatrix.FunctionOnlyAlgebra`. The following families separate matrix shape from the Bernstein grid that remains attached to the result.

4.6.1 Shape Inspection And Identity

Syntax

Call forms

```
sz = size(A)
sz = size(A, dim)
n = length(A)
n = height(A)
n = width(A)
n = numel(A)
n = ndims(A)
B = +A
tf = isequal(A, B, ...)
```

Arguments and options

- All shape queries report the two-dimensional matrix payload, never the physical-cell grid. In particular, `ndims(A)` is 2.
- Unary `+` returns an equivalent `pdmatrix`; it does not alter the stored coefficient evidence.
- `isequal` returns a logical scalar. Coefficient-backed objects are compared after compatible grid refinement and degree elevation; function-only objects compare their stored metadata and handle.

4.6.2 Transpose, Vectorization, Diagonals, And Reshape

Syntax

Call forms

```
B = transpose(A)
B = ctranspose(A)
B = A.'
B = A'
B = vec(A)
B = diag(A)
B = diag(A, k)
B = reshape(A, [m n])
B = reshape(A, m, n)
B = squeeze(A)
```

For finite real payloads, `transpose` and `ctranspose` have the same numerical effect. `vec` uses MATLAB column-major vectorization. `diag` extracts a diagonal, or constructs a square diagonal matrix from a vector payload; its finite integer `k` may not select an empty diagonal. `reshape` accepts exactly two positive dimensions, with at most one inferred `[]` dimension, and preserves the matrix element count. `squeeze` retains the package's two-dimensional matrix convention.

Invalid diagonal offsets or empty diagonal selections raise `pdmat:InvalidDiag`; malformed or size-changing reshape requests raise `pdmat:InvalidReshape`. These transformations require coefficient evidence and otherwise raise `pdmat:FunctionOnlyAlgebra`.

Example

These structural methods return new `pdmat` objects whose matrix size changes while the grid and degree metadata remain attached to the coefficient storage.

MATLAB input

```
A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
V = vec(A)
size(V)
D = diag(A)
size(D)
R = reshape(A, [1 4])
size(R)
```

Output

```
V =
  pdmat [4 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     4     1

D =
```

```

pdmat [2 1] over 1-D grid, degree 1, source coefficient-backed

ans =
     2     1

R =
pdmat [1 4] over 1-D grid, degree 1, source coefficient-backed

ans =
     1     4

```

4.6.3 Triangular Parts And Reductions

Syntax

Call forms

```

B = tril(A)
B = tril(A, k)
B = triu(A)
B = triu(A, k)
B = trace(A)
B = sum(A)
B = sum(A, dim)
B = sum(A, "all")
B = mean(A)
B = mean(A, dim)
B = mean(A, "all")
B = cumsum(A)
B = cumsum(A, dim)

```

`tril` and `triu` retain the lower or upper part of each payload; `k` is a finite integer diagonal offset. `trace` returns a 1-by-1 `pdmat`. For `sum`, `mean`, and `cumsum`, `dim` is a positive integer matrix dimension; "all" is additionally accepted by `sum` and `mean`.

Invalid triangular offsets raise `pdmat:InvalidTriangularPart`. Invalid call forms for `sum`, `mean`, or `cumsum` raise `pdmat:InvalidSum`, `pdmat:InvalidMean`, or `pdmat:InvalidCumsum`. These coefficient transformations also reject function-only objects with `pdmat:FunctionOnlyAlgebra`.

Example

Summary methods follow the same coefficient-wise rule.

MATLAB input

```

A = pdmat({[0 1]}, {[1 2; 3 4], [10 20; 30 40]}, Degree=1);
Tr = trace(A)
H = [diag(A), diag(A)]
size(Tr)
size(H)

```

Output

```
Tr =  
  pdmat [1 1] over 1-D grid, degree 1, source coefficient-backed  
  
H =  
  pdmat [2 2] over 1-D grid, degree 1, source coefficient-backed  
  
ans =  
      1      1  
  
ans =  
      2      2
```

4.6.4 Reordering And Repetition

Syntax

Call forms

```
B = flip(A)  
B = flip(A, dim)  
B = flipud(A)  
B = fliplr(A)  
B = rot90(A)  
B = rot90(A, k)  
B = repmat(A, m)  
B = repmat(A, m, n)  
B = repmat(A, [m n])
```

`flip` uses MATLAB's first nonsingleton matrix dimension by default; `flipud` and `fliplr` select rows and columns. `rot90` defaults to one quarter-turn and accepts finite integer `k`. `repmat` accepts a positive scalar `m`, expanded to `[m m]`, two positive integer repetition counts, or a two-element count vector. These transformations leave the physical grid and Bernstein degree unchanged.

Invalid flip dimensions, quarter-turn counts, or repetition shapes raise `pdmat:InvalidFlip`, `pdmat:InvalidRot90`, or `pdmat:InvalidRepmat`. A function-only object raises `pdmat:FunctionOnly` Algebra for coefficient-transforming forms.

4.6.5 Block Assembly

Syntax

Call forms

```
C = horzcat(A, B, ...)  
C = vertcat(A, B, ...)  
C = [A, B]  
C = [A; B]  
C = cat(dim, A, B, ...)  
C = blkdiag(A, B, ...)
```

The bracket forms call `horzcat` and `vertcat`; `cat` supports only dimensions 1 and 2. Every nonnumeric operand must be coefficient-backed, and at least one operand of `cat` or `blkdiag` must be a `pdmatrix`. Finite real numeric blocks are promoted on the common grid. Compatible grids with the same bounds are merged and lower-degree operands are elevated before their coefficient payloads are assembled.

Incompatible block sizes, operands, or `cat` syntax raise `pdmatrix:InvalidConcatenation`; dimensions other than 1 or 2 raise `pdmatrix:UnsupportedCatDimension`. Invalid `blkdiag` inputs raise `pdmatrix:InvalidBlkdiag`. Function-only operands raise `pdmatrix:FunctionOnlyAlgebra`, and incompatible parameter bounds raise `pdmatrix:MixedGrid`.

Example

MATLAB input

```
A = pdmatrix({[0 1]}, {1, 2}, Degree=1);  
size([A, A])  
size([A; A])  
size(blkdiag(A, 5))
```

Output

```
ans =  
    1    2  
  
ans =  
    2    1  
  
ans =  
    2    2
```

4.6.6 Equality And MATLAB Indexing Protocol

`isequal(A,B,...)` compares coefficient evidence rather than point samples. The `end(A,k,n)` and `numArgumentsFromSubscript(A,S,ctx)` methods support ordinary two-index expressions and dot access. Matrix slicing and assignment are documented in section 4.7; `subsref` and `subsasgn` are MATLAB protocol methods, not a second storage API.

Syntax

Call forms

```
tf = isequal(A, B, ...)
idx = end(A, k, n)
n = numArgumentsFromSubscript(A, S, ctx)
B = A(1:end, end)
```

4.7 Indexing And Assignment

Syntax

Call forms

```
B = A(rows, cols)
value = A.PropertyName
A(rows, cols) = numericValue
A(rows, cols) = B
```

Arguments and options

- `rows` and `cols` are ordinary MATLAB matrix subscripts, such as `:`, `end`, `1`, or `2:3`.
- `PropertyName` is a public property or method name for dot access.
- `numericValue` is a finite real matrix with the selected block size; `B` is a compatible coefficient-backed `pdmat` block.
- `value` is the property or method result. `B` is a sliced `pdmat`; assignment updates every local coefficient and elevates degrees when required.

Example

Indexing returns a smaller `pdmat`; assignment updates the corresponding matrix entries in every local coefficient.

MATLAB input

```
A = pdmat({[0 1]}, {[1 2 3; 4 5 6], [10 20 30; 40 50 60]}, Degree=1);
lastCol = A(:, end)
topTail = A(1, 2:3)
A(:, 2) = 5;
A
```


Output

```
lastCol =  
  pdmat [2 1] over 1-D grid, degree 1, source coefficient-backed  
  
topTail =  
  pdmat [1 2] over 1-D grid, degree 1, source coefficient-backed  
  
A =  
  pdmat [2 3] over 1-D grid, degree 1, source coefficient-backed
```

The assignment does not change the object degree because the numeric value is promoted to degree-one coefficient data before insertion.

Validation and errors

Invalid row or column selectors raise `pdmat:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdmat:UnsupportedAssignment`; an incompatible right-hand side raises `pdmat:InvalidAssignment`. Slicing or assignment on a function-only object raises `pdmat:FunctionOnlyAlgebra`.

4.8 Limits

- Function-only objects created without `Degree` do not have Bernstein coefficient evidence for `bernsteinTable` or coefficient algebra.
- Numeric operands must be finite real nonempty matrices with compatible sizes, or finite real scalars that can broadcast to the required size.
- Grid refinement is supported for matching parameter dimensions and matching grid bounds.

4.8.1 See Also

`pdmat`, `evaluate`, `bernsteinTable`, `plot`, `pdbase`.

5

pdvar Reference

`pdvar` creates continuous cell-local Bernstein decision expressions backed by YALMIP `sdpvar` coefficients. It participates in supported affine and known-data algebra. It does not choose solvers or call `optimize`; solver-facing work happens after comparison overloads create `pdlmi` objects and `toYalmip` converts them to YALMIP constraints.

5.1 Lookup

Task	Entry
Construct decisions	<code>pdvar</code>
Inspect coefficients	<code>bernsteinTable</code>
Convert assigned coefficients	<code>value</code>
Differentiate rates	<code>rhodiff</code>
Affine/mixed algebra	<code>+</code> , <code>-</code> , <code>*</code> with known data
Matrix/index methods	<code>structural methods</code> , <code>indexing</code> and <code>assignment</code>
Build LMIs	<code><=</code> , <code>>=</code>

5.2 Construction

Syntax

Call forms

```
P = pdvar(n, gridVectors)
P = pdvar(n, n, gridVectors)
P = pdvar(n, m, gridVectors)
P = pdvar(..., "full")
P = pdvar(..., "symmetric")
P = pdvar(..., Degree=0)
P = pdvar(..., Degree=1)
P = pdvar(..., Degree=d)
P = pdvar(..., RateBounds=rb)
```

Arguments and options

- `pdvar(n, gridVectors)` creates an $n \times n$ square variable using scalar-size shorthand; for example, `pdvar(2, {[0 1]})`. Square variables default to symmetric YALMIP coefficients.
- `pdvar(n, n, gridVectors)` is the explicit square-size form; for example, `pdvar(2, 2, {[0 1]})`. It has the same default symmetric structure as the shorthand.
- `pdvar(n, m, gridVectors)` creates an $n \times m$ variable. Rectangular variables default to full YALMIP coefficients; for example, `pdvar(2, 3, {[0 1]})`.
- "full" and "symmetric" explicitly choose the YALMIP coefficient structure. "symmetric" requires a square size.
- `Degree=1` is the default continuous Bernstein decision variable. Every finite nonnegative integer `Degree=d` is supported. `Degree=0` creates one parameter-independent decision matrix shared across the grid; `d >= 1` creates continuous piecewise Bernstein data whose neighboring cells share complete control faces.
- `RateBounds=rb` stores parameter-rate metadata for later `rhodiff(P)` calls. Use `RateBounds=[-1 1]` for one parameter; use a two-row matrix for two parameters, as shown in the rate-metadata example below.

Example

The square shorthand creates a 2×2 continuous decision object. The default degree is one and the object carries no rate metadata.

MATLAB input

```
yal mip('clear')
Ps = pdvar(2, {[0 1]});
class(Ps)
size(Ps)
Ps.Degree
Ps.HasRateDependence
```

Output

```
ans =
    'pdvar'

ans =
     2     2

ans =
     1

ans =
    logical
     0
```

Use two dimension arguments for a rectangular full variable.

MATLAB input

```
yalmip('clear')
Pf = pdvar(2, 3, {[0 1]});
class(Pf)
size(Pf)
Pf.Degree
```

Output

```
ans =
    'pdvar'

ans =
     2     3

ans =
     1
```

The structure flag makes the coefficient structure explicit. This is useful when a square variable should use full, not symmetric, coefficients.

MATLAB input

```
yalmip('clear')
Pfull = pdvar(2, 2, {[0 1]}, "full");
class(Pfull)
Pfull.MatrixSize
Pfull.Degree
```

Output

```
ans =
    'pdvar'

ans =
     2     2

ans =
     1
```

Use Degree=0 for a parameter-independent decision variable stored with grid metadata.

MATLAB input

```
yalmip('clear')
P0 = pdvar(1, [0 1 2], Degree=0);
class(P0)
P0.Degree
P0.ncell()
```

Output

```
ans =  
    'pdvar'  
  
ans =  
    0  
  
ans =  
    2
```

Rate bounds are metadata for later `rhodiff(P)` calls. `pdlmi` constrains rate vertices only after an expression such as `rhodiff` has stored rate-vertex rows.

MATLAB input

```
yalmip('clear')  
Pr = pdvar(1, {[0 1], [10 20]}, RateBounds=[-1 2; -3 4]);  
class(Pr)  
Pr.HasRateDependence  
Pr.RateBounds
```

Output

```
ans =  
    'pdvar'  
  
ans =  
    logical  
         1  
  
ans =  
    -1     2  
    -3     4
```

Validation and errors

Missing dimensions, a missing grid, or a grid in the wrong argument position raises `pdvar:InvalidInput`. Nonpositive or noninteger matrix sizes raise `pdvar:InvalidMatrixSize`; malformed degrees raise `pdvar:InvalidDegree`. Malformed option sequences raise `pdvar:InvalidOptions`; an unknown option raises `pdvar:UnknownOption`; attempts to set subclass-owned metadata raise `pdvar:UnsupportedOption`. A symmetric request for a nonsquare matrix raises `pdvar:InvalidStructure`. Grid validation uses `pdvar:InvalidGrid` and `pdvar:InvalidGridVector`.

`RateBounds` is validated by the inherited storage contract and must be a finite ℓ -by-2 matrix with lower bounds no greater than upper bounds. Constructor violations therefore raise `pdbase:InvalidRateBounds`.

5.3 rhodiff

Syntax

Call forms

```
D = rhodiff(P)
D = rhodiff(P, rb)
```

Arguments and options

- `P` is a `pdvar` without existing rate-vertex rows.
- `rb` is a finite ℓ -by-2 numeric matrix of lower and upper parameter-rate bounds, with each lower bound no greater than its upper bound. It must equal `P.RateBounds` when the object already carries nonempty bounds.
- Without `rb`, `rhodiff(P)` requires nonempty `P.RateBounds`.
- `D` is a discontinuous `pdvar` with one coefficient row per rate vertex. In one parameter a degree- m derivative has degree $m - 1$, while a degree-zero derivative remains zero; multivariate partials are elevated into the implementation's common tensor degree.

For a one-parameter degree-one coefficient pair P_0, P_1 on a cell of width h , the rate-dependent derivative row is

$$\dot{P}(\rho, \dot{\rho}) = \dot{\rho} \frac{P_1 - P_0}{h}.$$

The implementation stores one row for each allowed rate vertex.

Validation and errors

Unsupported call arity or differentiating an expression that already contains rate-vertex rows raises `pdvar:InvalidDiff`. Explicit malformed rate bounds raise `pdvar:InvalidRateBounds`. Calling `rhodiff(P)` without stored bounds raises `pdvar:MissingRateBounds`; explicit bounds that disagree with `P.RateBounds` raise `pdvar:RateBoundsMismatch`.

Example

When rate bounds live on the object, `rhodiff(P)` uses them directly.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1 2]}, "symmetric", RateBounds=[-1 1]);
D = rhodiff(P);
class(D)
D.Degree
D.IsContinuous
size(D.coeffs(1))
D.RateBounds
```

Output

```
ans =
    'pdvar'

ans =
     0

ans =
    logical
     0

ans =
     2     1

ans =
    -1     1
```

For a two-parameter object, the rate-vertex rows enumerate all lower/upper combinations from the two rows of `RateBounds`.

MATLAB input

```
yalmip('clear')
Q = pdvar(1, {[0 1], [10 20]}, RateBounds=[-1 1; -2 2]);
E = rhodiff(Q);
class(E)
E.Degree
size(E.coeffs([1 1]))
E.RateBounds
```

Output

```
ans =  
    'pdvar'  
  
ans =  
    1  
  
ans =  
    4    4  
  
ans =  
    -1    1  
    -2    2
```

5.4 bernsteinTable

Syntax

Call forms

```
T = bernsteinTable(P)  
T = bernsteinTable(P, cellSubs)  
T = bernsteinTable(P, "oneLine")  
T = bernsteinTable(P, cellSubs, "oneLine")
```

Arguments and options

- `P` is a `pdvar` object.
- `cellSubs` selects one physical cell; omit it to list every cell.
- `"oneLine"` is the only supported text option. It returns one compact Bernstein expression per selected cell.
- `T` is a table. Ordinary rows contain the local label, basis, physical-node flag, and symbolic or numeric value. A rate-dependent derivative also includes `RateVertexIndex` and `RateVertex`.

Example

MATLAB input

```
yal mip('clear')  
P = pdvar(1, {[0 1]});  
T = bernsteinTable(P, "oneLine");  
disp(T)
```


Output

CellSubscript	Expression
-----	-----
{[1]}	"(1-alpha)*internal(1) + alpha*internal(2)"

Validation and errors

`bernsteinTable` is an inspection method, not a solver call. Text options other than `"oneLine"`, repeated selectors, invalid physical-cell selectors, or inconsistent rate-vertex rows raise `pdvar:InvalidBernsteinTableInput`.

5.5 Affine And Mixed Algebra

Syntax

Call forms

```
R = P + Q
R = P + A
R = A + P
R = P + M
R = M + P
R = P - Q
R = P - A
R = A - P
R = +P
R = -P
R = A * P
R = P * A
R = M * P
R = P * M
```

Arguments and options

- `pdvar`: affine addition and subtraction with compatible size, grid, and rate metadata.
- Coefficient-backed `pdmat`: mixed known/decision algebra for ordinary `pdvar` expressions, including products where only one side contains decisions.
- Numeric scalar or matrix: promoted as constant coefficient data for ordinary `pdvar` expressions when finite real and size-compatible.
- Affine `sdpvar` matrix: promoted as parameter-independent affine data when real, linear, and size-compatible.

- Rate-vertex derivative rows: supported in the tested derivative and known-data combinations used by `rhodiff` LMI residuals.
- Metadata-only rate-dependent `pdvar` products are unsupported until `rhodiff` has materialized rate-vertex rows.

Example

The affine sum with a numeric matrix keeps the same grid and degree.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
S = P + eye(2);
class(S)
size(S)
S.Degree
S.ContainsDecision
```

Output

```
ans =
    'pdvar'

ans =
     2     2

ans =
     1

ans =
    logical
     1
```

Known-data products are allowed when only one side contains decisions. The product degree is the sum of the known-data degree and the decision degree.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
A = pdmat({[0 1]}, {eye(2), 2*eye(2)}, Degree=1);
K = A * P;
R = P * A;
class(K)
class(R)
K.Degree
R.Degree
```

Output

```
ans =  
    'pdvar'  
  
ans =  
    'pdvar'  
  
ans =  
    2  
  
ans =  
    2
```

5.5.1 Operator-level entries

plus. Forms an affine sum of compatible `pdvar`, numeric, affine `sdpvar`, or coefficient-backed `pdmat` operands.

Syntax

Call forms

```
R = P + Q  
R = P + A  
R = A + P  
R = P + M  
R = M + P
```

minus. Forms an affine difference and preserves operand order; the same promotion and grid compatibility rules as `plus` apply.

Syntax

Call forms

```
R = P - Q  
R = P - A  
R = A - P  
R = P - M  
R = M - P
```

uplus and uminus. Unary plus preserves the expression; unary minus negates every coefficient expression.

Syntax

Call forms

```
R = +P  
R = -P
```

mtimes. Supports products with known numeric or coefficient data when decision dependence occurs on at most one side. Products of two decision expressions, or products with rate dependence on both sides, are rejected to preserve affine modeling.

Syntax

Call forms

```
R = A * P  
R = P * A  
R = M * P  
R = P * M
```

Example

MATLAB input

```
yalmp('clear')  
P = pdvar(1, {[0 1]});  
class(+P)  
class(-P)  
size(2 * P)
```

Output

```
ans =  
    'pdvar'  
  
ans =  
    'pdvar'  
  
ans =  
     1     1
```

Validation and errors

Incompatible operands for affine $+$ and $-$ raise `pdvar:InvalidAddition` or `pdvar:InvalidSubtraction`. Products with incompatible dimensions, two decision-bearing factors, nonlinear symbolic data, unsupported rate metadata, or other non-affine combinations raise `pdvar:InvalidMultiplication`. Incompatible parameter dimensions or outer bounds raise `pdvar:MixedGrid`. A function-only `pdmat` operand raises `pdvar:FunctionOnlyAlgebra`.

5.6 Matrix Methods

Matrix methods act on each local YALMIP-backed payload while preserving compatible grid, degree, and rate metadata. This is matrix structure, not parameter-grid manipulation. The families below use the same organization as the `pdmat` reference, but their outputs remain `pdvar` expressions and supported assembly may additionally use coefficient-backed `pdmat`, affine `sdpvar`, and numeric blocks.

5.6.1 Shape Inspection And Identity

Syntax

Call forms

```
sz = size(P)
sz = size(P, dim)
n = length(P)
n = height(P)
n = width(P)
n = numel(P)
n = ndims(P)
Q = +P
tf = isequal(P, R, ...)
```

Arguments and options

- Shape queries report the two-dimensional matrix payload, never the physical-cell grid. In particular, `ndims(P)` is 2.
- Unary `+` preserves the decision expression and its coefficient graph.
- `isequal` returns a logical scalar only when the grid, degree, matrix size, metadata, and local YALMIP payloads are identical; it does not merge grids or elevate degrees.

5.6.2 Transpose, Vectorization, Diagonals, And Reshape

Syntax

Call forms

```
Q = transpose(P)
Q = ctranspose(P)
Q = P.'
Q = P'
Q = vec(P)
Q = diag(P)
Q = diag(P, k)
Q = reshape(P, [m n])
Q = reshape(P, m, n)
Q = squeeze(P)
```

`transpose` and `ctranspose` transpose every local decision payload without breaking its YALMIP coefficient graph. `vec`, `diag`, and `reshape` have the same matrix meanings as their `pdmat` counterparts: `k` is a finite integer diagonal offset, and `reshape` uses exactly two positive dimensions with at most one inferred `[]` dimension. `squeeze` retains the two-dimensional matrix convention.

Invalid diagonal offsets or empty diagonal selections raise `pdvar:InvalidDiag`; malformed or size-changing reshape requests raise `pdvar:InvalidReshape`.

Example

Shape-changing operations return new `pdvar` objects and preserve the decision-backed coefficient graph.

MATLAB input

```
yalmip('clear')
P = pdvar(2, 3, {[0 1]}, "full");
Tt = P.';
class(Tt)
size(Tt)
V = vec(P);
class(V)
size(V)
R = reshape(P, [3 2]);
class(R)
size(R)
```

Output

```
ans =
    'pdvar'

ans =
     3     2

ans =
    'pdvar'

ans =
     6     1

ans =
```

```
'pdvar'  
ans =  
    3    2
```

5.6.3 Triangular Parts And Reductions

Syntax

Call forms

```
Q = tril(P)  
Q = tril(P, k)  
Q = triu(P)  
Q = triu(P, k)  
Q = trace(P)  
Q = sum(P)  
Q = sum(P, dim)  
Q = sum(P, "all")  
Q = mean(P)  
Q = mean(P, dim)  
Q = mean(P, "all")  
Q = cumsum(P)  
Q = cumsum(P, dim)
```

tril and **triu** use a finite integer diagonal offset. **trace** returns a 1-by-1 decision expression. For **sum**, **mean**, and **cumsum**, **dim** is a positive integer matrix dimension; "all" is additionally accepted by **sum** and **mean**.

Invalid triangular offsets raise `pdvar:InvalidTriangularPart`. Invalid call forms for **sum**, **mean**, or **cumsum** raise `pdvar:InvalidSum`, `pdvar:InvalidMean`, or `pdvar:InvalidCumsum`.

Example

Summary methods use the same local-payload mechanism.

MATLAB input

```
yalmip('clear')  
P = pdvar(2, 3, {[0 1]}, "full");  
S = sum(P, 2);  
B = blkdiag(P(1:2,1:2), eye(2));  
class(S)  
class(B)  
size(S)  
size(B)
```

Output

```
ans =  
    'pdvar'  
  
ans =  
    'pdvar'  
  
ans =  
     2     1  
  
ans =  
     4     4
```

5.6.4 Reordering And Repetition

Syntax

Call forms

```
Q = flip(P)  
Q = flip(P, dim)  
Q = flipud(P)  
Q = fliplr(P)  
Q = rot90(P)  
Q = rot90(P, k)  
Q = repmat(P, m)  
Q = repmat(P, m, n)  
Q = repmat(P, [m n])
```

`flip` defaults to MATLAB's first nonsingleton matrix dimension; `flipud` and `fliplr` select rows and columns. `rot90` accepts a finite integer quarter-turn count, and `repmat` accepts a positive scalar `m`, expanded to `[m m]`, two positive integer repetition counts, or a two-element count vector. These methods preserve the local decision expressions and compatible parameter metadata.

Invalid flip dimensions, quarter-turn counts, or repetition shapes raise `pdvar:InvalidFlip`, `pdvar:InvalidRot90`, or `pdvar:InvalidRepmat`.

5.6.5 Block Assembly

Syntax

Call forms

```
Q = horzcat(P, R, ...)  
Q = vertcat(P, R, ...)  
Q = [P, R]  
Q = [P; R]  
Q = cat(dim, P, R, ...)  
Q = blkdiag(P, R, ...)
```

The bracket forms concatenate compatible `pdvar`, coefficient-backed `pdmat`, affine `sdpvar`, and finite real numeric blocks. `cat` supports only matrix dimensions 1 and 2. The operands are aligned on a compatible grid, and degree/rate metadata are propagated only when that combination remains valid for an affine decision expression.

Incompatible block sizes, operands, rate metadata, or `cat` syntax raise `pdvar:InvalidConcatenation`; dimensions other than 1 or 2 raise `pdvar:UnsupportedCatDimension`. Invalid `blkdiag` inputs raise `pdvar:InvalidBlkdiag`. Incompatible parameter bounds raise `pdvar:MixedGrid`; a function-only `pdmat` block raises `pdvar:FunctionOnlyAlgebra`.

Example

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "full");
size([P, P])
size([P; P])
size(blkdiag(P, eye(2)))
```

Output

```
ans =
     2     4

ans =
     4     2

ans =
     4     4
```

5.6.6 Equality And MATLAB Indexing Protocol

`isequal(P,R,...)` is an exact identity check: it requires equal grid, degree, matrix size, metadata, and local YALMIP payloads, without grid merging or degree elevation. `end(P,k,n)` and `numArgumentsFromSubscript(P,S,ctx)` support ordinary two-index matrix expressions and dot access. Matrix slicing and assignment are documented in section 5.7; `subsref` and `subsasgn` implement that MATLAB protocol rather than an additional storage interface.

Syntax

Call forms

```
tf = isequal(P, R, ...)
idx = end(P, k, n)
n = numArgumentsFromSubscript(P, S, ctx)
Q = P(1:end, end)
```

5.7 Indexing And Assignment

Syntax

Call forms

```
Q = P(rows, cols)
value = P.PropertyName
P(rows, cols) = numericValue
P(rows, cols) = Q
P(rows, cols) = A
P(rows, cols) = X
```

Arguments and options

- `rows` and `cols` are ordinary MATLAB matrix subscripts, such as `:`, `end`, `1`, or `1:2`.
- `PropertyName` is a public property or method name for dot access.
- `numericValue`, `Q`, `A`, and `X` are, respectively, a finite real matrix, compatible `pdvar`, coefficient-backed `pdmat`, or affine `sdpvar` block matching the selection.
- `value` is the property or method result. `Q` is a sliced `pdvar`; assignment may elevate degrees and propagate compatible rate metadata across every physical cell and local coefficient.

Example

Indexing returns a smaller `pdvar`; assignment updates the selected matrix entries across every local coefficient.

MATLAB input

```
yalmp('clear')
P = pdvar(2, 3, {[0 1]}, "full");
col = P(:, 2);
P(:, 1) = 0;
class(col)
size(col)
size(P)
```

Output

```
ans =  
    'pdvar'  
  
ans =  
     2     1  
  
ans =  
     2     3
```

Validation and errors

Invalid row or column selectors raise `pdvar:InvalidSubscript`. Linear indexing, deletion, or non-parenthesis assignment raises `pdvar:UnsupportedAssignment`; an incompatible or non-affine right-hand side raises `pdvar:InvalidAssignment`.

5.8 value

Syntax

Call forms

```
A = value(P)  
rows = value(rhodiff(P))
```

`value` converts assigned `pdvar` coefficients into known, coefficient-backed `pdmat` data. An ordinary expression returns one `pdmat`, preserving its grid, matrix size, degree, local coefficient order, and continuity metadata. A rate-vertex expression created by `rhodiff` returns a $1\text{-by-}2^\ell$ cell array of `pdmat` objects in `helper.combRows(RateBounds)` lower/upper vertex order. The returned `pdmat` objects do not carry rate metadata. Every symbolic coefficient must have an assigned finite real YALMIP value; otherwise the method raises `pdvar:UnassignedValue`.

Example

MATLAB input

```
yalmip('clear')  
P = pdvar(1, [0 1], Degree=2);  
c = P.coeffs(1);  
assign([c{:}], [1 2 4]);  
A = value(P);  
A.coeffs(1)
```

Output

```
ans =  
    {[1]}    {[2]}    {[4]}
```

5.9 Comparisons To pdlmi

Syntax

Call forms

```
C = P <= 0  
C = P >= 0  
C = lhs <= rhs  
C = lhs >= rhs
```

Arguments and options

- **P** and **lhs** are square, coefficient-wise symmetric or Hermitian **pdvar** expressions after residual assembly.
- **rhs** is commonly 0; compatible numeric, **pdmat**, affine **sdpvar**, and **pdvar** expressions are also accepted.
- **C** is a **pdlmi** wrapper. The comparison creates direct coefficient constraints but does not solve an optimization problem.

Example

Comparison creates a **pdlmi** wrapper with one stored constraint for each physical cell and local Bernstein coefficient.

MATLAB input

```
yalmp('clear')  
P = pdvar(2, {[0 0.5 1]}, "symmetric");  
Cneg = P <= 0;  
class(Cneg)  
numel(Cneg.Constraints)  
Cpos = P >= 0;  
class(Cpos)  
numel(Cpos.Constraints)
```

Output

```
ans =  
  'pdlmi'  
  
ans =  
  4  
  
ans =  
  'pdlmi'  
  
ans =  
  4
```

5.10 Limits

- Constructor-created `pdvar` objects support every finite nonnegative integer degree. Higher degree increases the number of Bernstein control matrices per cell.
- Products may contain decision variables on at most one side; nonlinear decision products such as $P * Q$ are outside this slice.
- Function-only `pdmat` operands cannot enter `pdvar` algebra unless they were constructed with explicit Bernstein evidence.
- `rhodiff` of an existing rate-vertex derivative expression is not part of the current public workflow.

5.10.1 See Also

`pdvar`, `rhodiff`, `bernsteinTable`, `pdlmi`, `pdmat`.

6

pdlmi Reference

pdlmi stores YALMIP constraints assembled from square symmetric `pdvar` expressions. Direct coefficient-wise assembly is the default; cell-local Pólya elevation and the full box Bernstein–Gram preordering are explicit alternatives. It is the package boundary between Bernstein objects and ordinary YALMIP solver calls.

6.1 Lookup

Task	Entry
Construct wrapper	<code>pdlmi</code> , <code><=/>=</code>
Count constraints	<code>direct coefficient constraints</code>
Select Pólya elevation	<code>UsePolya</code> , <code>PolyaDegree</code> , and <code>applyPolya</code>
Select full box preordering	<code>UseFullBoxPreorder</code> , <code>FullBoxOrder</code> , and <code>applyFullBoxPreorder</code>
Export to YALMIP	<code>toYalmip</code>
Solve	<code>solver-facing use</code> , <code>Examples</code>

6.2 Construction And Comparisons

Syntax

Call forms

```
C = pdlmi(expr, relation)
C = pdlmi(expr, relation, "UsePolya")
C = pdlmi(expr, relation, UsePolya=true, PolyaDegree=d)
C = pdlmi(expr, relation, "UsePolya", "PolyaDegree", d)
C = pdlmi(expr, relation, "UseFullBoxPreorder")
C = pdlmi(expr, relation, UseFullBoxPreorder=true)
C = pdlmi(expr, relation, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=true, FullBoxOrder=r)
C = pdlmi(expr, relation, UseFullBoxPreorder=false, FullBoxOrder=r)
C = lhs <= rhs
C = lhs >= rhs
```

Arguments and options

- `expr` is a square `pdvar` expression whose coefficient matrices are symmetric or Hermitian; `relation` is exactly "`<=`" or "`>=`".
- `UsePolya` is a logical scalar option with default `false`. Its bare form, "`UsePolya`", and `UsePolya=true` without a degree select increment one.
- `PolyaDegree=d` is a finite nonnegative integer scalar. It elevates the residual by `d` in every parameter direction before assembling constraints. Supplying it without `UsePolya` enables Pólya and issues `pdlmi:ImplicitUsePolya`; `UsePolya=false` with positive `d` raises `pdlmi:ConflictingPolyaOptions`.
- `UseFullBoxPreorder` is a logical scalar option with default `false`. Its bare form and the explicit value `true` select full-box assembly at the smallest admissible order.
- `FullBoxOrder=r` is a finite nonnegative integer scalar and is an absolute order, not a degree increment. Supplying it without `UseFullBoxPreorder` enables full-box assembly. When full-box assembly is enabled, the minimum is $\lfloor m/2 \rfloor$ for a one-parameter residual of degree m , and $\lceil m/2 \rceil$ for two or more parameters. Explicitly pairing `UseFullBoxPreorder=false` with `FullBoxOrder=r` suppresses that automatic selection: `C` remains direct and records `r` only as inspectable metadata; the order is not applied to assembly.
- Pólya and full-box selection are mutually exclusive. Enabling both raises `pdlmi:ConflictingRelaxations`.
- `C` is direct coefficient-wise assembly unless one opt-in mode is selected. Comparisons such as `lhs >= rhs` are the ordinary user-facing entry point and accept compatible numeric, `pdmat`, affine `sdpvar`, and `pdvar` right-hand sides.

The inspectable properties `UsePolya`, `PolyaDegree`, `UseFullBoxPreorder`, and `FullBoxOrder` report the selected mode and order metadata. Their set access is private. Comparison-created direct objects, and direct constructor calls that omit `FullBoxOrder`, have both selectors false and both numeric properties zero. The explicit-false combination described above is the exception: it retains `FullBoxOrder=r` while `UseFullBoxPreorder` remains false and the stored constraints remain direct.

Example

The direct constructor is equivalent to the comparison path when the relation and options are explicit.

MATLAB input

```
yalmip('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = pdlmi(P, ">=", UsePolya=false, PolyaDegree=0);
class(C)
numel(C.Constraints)
C.UsePolya
C.PolyaDegree
C.UseFullBoxPreorder
C.FullBoxOrder
```

Output

```
ans =
    'pdlmi'

ans =
     2

ans =
    logical
     0

ans =
     0

ans =
    logical
     0

ans =
     0
```

Validation and errors

The constructor rejects a non-`pdvar` residual with `pdlmi:InvalidExpression`, a relation other than `"<="` or `">="` with `pdlmi:InvalidRelation`, and malformed, unknown, or duplicate options with `pdlmi:InvalidOptions`, `pdlmi:UnknownOption`, or `pdlmi:DuplicateOption`. Nonlogical selectors raise `pdlmi:InvalidUsePolya` or `pdlmi:InvalidUseFullBoxPreorder`. Malformed Pólya increments and full-box orders raise `pdlmi:InvalidPolyaDegree` and `pdlmi:InvalidFullBoxOrder`, respectively. When full-box assembly is enabled, a valid integer order below the dimension-dependent minimum instead raises `pdlmi:FullBoxOrderTooLow`. An explicitly false selector keeps assembly direct, so this admissibility minimum is not applied to its retained order metadata. Every assembly path retains the square and symmetry checks, which raise `pdlmi:InvalidMatrixSize` and `pdlmi:NonSymmetricExpression`.

6.3 Direct Coefficient Assembly

For each physical cell, local Bernstein coefficient, and rate-vertex row if present, `pdlmi` creates one YALMIP semidefinite constraint. For an ordinary expression with N_c physical cells and N_b Bernstein coefficients per cell, the number of stored constraints is $N_c N_b$. For a `rhodiff` expression with N_v rate vertices, the count becomes $N_c N_b N_v$.

6.4 Opt-In Pólya Assembly And `applyPolya`

With `UsePolya=true`, `pdlmi` first elevates the stored residual by `PolyaDegree` in every parameter direction, then constrains every elevated local coefficient and every active rate row. Thus, if the original degree is m , the count is $N_c(m + d + 1)^\ell$ for ordinary expressions and $N_c(m + d + 1)^\ell N_v$ for rate-vertex expressions. An increment of zero is valid and has the same coefficient count as direct assembly.

Syntax

Call forms

```
Cpolya = C.applyPolya()  
Cpolya = C.applyPolya(d)
```

`applyPolya` is a value-class method: it returns a new `pdlmi`, leaves `C` unchanged, and rebuilds from `C.Residual`. Selecting a new increment therefore replaces rather than compounds a previous selection. The no-argument form uses one; an invalid increment raises `pdlmi:InvalidPolyaDegree`.

Example

MATLAB input

```
yal mip('clear')  
P = pdvar(1, {[0 1]}, Degree=1);  
direct = P >= 0;  
polya = direct.applyPolya(2);  
direct.PolyaDegree  
polya.UsePolya  
polya.PolyaDegree  
numel(polya.Constraints)
```

Output

```
ans =
    0

ans =
    logical
    1

ans =
    2

ans =
    4
```

6.5 Full Box Bernstein–Gram Preordering And `applyFullBoxPreorder`

The full-box mode is a cell-local positive-semidefinite representation in the forward local coordinates $\alpha_s \in [0, 1]$. It is a box-specific Bernstein–Gram preordering, not a general Putinar certificate, general-domain SOS interface, or full-space SOS representation. It adds no implicit positivity margin.

Syntax

Call forms

```
Cbox = C.applyFullBoxPreorder()
Cbox = C.applyFullBoxPreorder(r)
```

The no-argument form selects the smallest admissible absolute order. The explicit form selects `r`; it does not add `r` to the residual degree. Both forms are value-class selectors: they return a new `pdlmi`, leave `C` unchanged, and rebuild from `C.Residual`. Reapplication therefore replaces an earlier full-box order or a Pólya selection rather than compounding it.

For a one-parameter residual of degree m , the minimum and matching target degree are parity-specific:

$$r_{\min} = \left\lfloor \frac{m}{2} \right\rfloor, \quad m_{\text{target}} = \begin{cases} 2r, & m \text{ even,} \\ 2r + 1, & m \text{ odd.} \end{cases}$$

For an n -by- n residual, define the lifted, binomially normalized basis in forward index order by

$$\mathcal{Z}_q(\alpha) = \begin{bmatrix} B_0^q(\alpha)I_n & \cdots & B_q^q(\alpha)I_n \end{bmatrix}^\top.$$

The positive-semidefinite target has the even-degree form

$$\mathcal{Z}_r(\alpha)^\top Q_0 \mathcal{Z}_r(\alpha) + \alpha(1 - \alpha) \mathcal{Z}_{r-1}(\alpha)^\top Q_1 \mathcal{Z}_{r-1}(\alpha),$$

where the second block is omitted at order zero. For odd degree it has the form

$$(1 - \alpha) \mathcal{Z}_r(\alpha)^\top Q_0 \mathcal{Z}_r(\alpha) + \alpha \mathcal{Z}_r(\alpha)^\top Q_1 \mathcal{Z}_r(\alpha).$$

Every present Gram matrix Q_i is constrained positive semidefinite.

For $\ell > 1$ parameters, the minimum is $r_{\min} = \lceil m/2 \rceil$, and the target has tensor degree $2r$ in every direction. For a tensor multi-degree $\mathbf{d}$, $\mathcal{Z}_{\mathbf{d}}(\boldsymbol{\alpha})$ denotes the tensor product of the normalized one-dimensional bases above, lifted by I_n in the same forward label order. One term is included for every mask $\boldsymbol{\mu} \in \{0, 1\}^\ell$:

$$\sum_{\boldsymbol{\mu} \in \{0, 1\}^\ell} \left(\prod_{s=1}^{\ell} [\alpha_s(1 - \alpha_s)]^{\mu_s} \right) \mathcal{Z}_{r\mathbf{1}-\boldsymbol{\mu}}(\boldsymbol{\alpha})^\top Q_{\boldsymbol{\mu}} \mathcal{Z}_{r\mathbf{1}-\boldsymbol{\mu}}(\boldsymbol{\alpha}).$$

A term whose tensor Gram degree has a negative component is omitted. For an n -by- n residual, a present mask has Gram dimension $n \prod_s (r - \mu_s + 1)$.

Assembly is independent for every physical cell and every active rate-vertex row; Gram variables are not shared across either boundary. For $\geq$, the method represents the residual. For $\leq$, it negates the target before forming the positive-semidefinite representation. Within each cell and row, the stored constraints are ordered as all PSD Gram blocks followed by exact equalities matching every Bernstein coefficient. Consequently, the number of stored constraints is not the direct coefficient count.

Validation and errors

Malformed orders raise `pdlmi:InvalidFullBoxOrder`; an admissible integer below the minimum raises `pdlmi:FullBoxOrderTooLow`. The constructor rejects malformed selectors with `pdlmi:InvalidUseFullBoxPreorder`, and simultaneous Pólya and full-box selection raises `pdlmi:ConflictingRelaxations`. The usual expression, relation, option, square-matrix, and symmetry checks described in section 6.2 apply unchanged.

6.6 toYalmip

Syntax

Call forms

```
F = toYalmip(C)
F = C.toYalmip()
```

Arguments and options

`toYalmip` concatenates the stored constraint cells into a YALMIP constraint array for direct, Pólya, and full-box objects. It has no package-specific options and does not add a solver wrapper, objective, margin, or diagnostic layer. The output is suitable for `optimize`, concatenation with other YALMIP constraints, or ordinary YALMIP inspection.

Example

The function-call and dot-call forms return equivalent YALMIP constraint arrays.

MATLAB input

```
yalmp('clear')
P = pdvar(2, {[0 1]}, "symmetric");
C = P >= 0;
class(C)
numel(C.Constraints)
F1 = toYalmip(C);
F2 = C.toYalmip();
isa(F1, "lmi") || isa(F1, "constraint")
length(F1)
isa(F2, "lmi") || isa(F2, "constraint")
length(F2)
```

Output

```
ans =
    'pdlmi'

ans =
     2

ans =
    logical
     1

ans =
     2

ans =
    logical
     1

ans =
     2
```

After this boundary, the package is no longer assembling Bernstein data; YALMIP owns solver choice, diagnostics, and optimization status.

6.7 See Also

pdvar, rhodiff, applyPolya, applyFullBoxPreorder, toYalmip, optimize.

6.8 Solver-Facing Use

`pdlmi` does not own solver settings or wrap `optimize`. After `toYalmip`, use ordinary YALMIP:

Call forms

```
opts = sdpsettings("solver", solverName, "verbose", 0);  
sol = optimize([C1.toYalmip, C2.toYalmip], objective, opts);
```

The solver selected by YALMIP must be available on the MATLAB path. The current test suite prefers MOSEK when available and otherwise falls back to LMILAB for smoke coverage.

7

Cross-Class Helper Reference

The `+helper` namespace contains exactly nine implementation utilities whose ownership genuinely crosses class boundaries. They are developer-facing building blocks, not an alternative modeling API: ordinary users should enter through `pdbase`, `pdmat`, `pdvar`, and `pdlmi`. Their signatures and current consumers are listed below so maintainers can distinguish shared contracts from class-local implementation details.

1. `helper.bernTbl(...)`
Builds ordinary, rate-vertex, or one-line Bernstein inspection tables with caller-supplied value and expression formatters. *Consumers:* `pdmat`, `pdvar`.
2. `helper.cellGet(vals, subs)`
Follows one tensor-subscript row through nested `LocalValues` and returns the selected leaf. *Consumers:* all four classes.
3. `helper.chk(val, id, msg, ...)`
Applies shared predicate tags and bounds, returning the input or raising the caller's error ID. *Consumers:* all four classes.
4. `helper.combRows(vecs)`
Returns Cartesian-product rows with earlier dimensions varying more slowly. *Consumers:* all four classes.
5. `helper.isZero(val, mode, ...)`
Classifies numeric, additive, nested-payload, or coefficient-backed object zero evidence. *Consumers:* `pdmat`, `pdvar`.
6. `helper.mapVals(vals, fcn, grid)`
Maps one function over every coefficient in a nested cell tree. *Consumers:* `pdmat`, `pdvar`.
7. `helper.matSubs(subs, sz, id)`
Normalizes numeric, logical, or colon matrix subscripts into row and column indices. *Consumers:* `pdmat`, `pdvar`.
8. `helper.mkGrid(grid, owner)`
Validates tensor-grid vectors and builds `GridInfo` with vectors, points, bounds, and node counts. *Consumers:* `pdbase`, `pdmat`, `pdvar`.
9. `helper.mkNest(counts, fcn[, prefix])`
Constructs recursive tensor storage and calls the leaf factory with each complete subscript row. *Consumers:* `pdbase`, `pdmat`, `pdvar`.

7.1 Shared Validation And Errors

`helper.chk` recognizes the predicate tags `numeric`, `real`, `cell`, `struct`, `nonempty`, `scalar`, `vector`, `matrix`, `finite`, `integer`, `positive`, `nonnegative`, `increasing`, and `rowbounds`; its named tests are `Size`, `Numel`, `MinNumel`, `Min`, and `Max`. Failed user-data checks raise the supplied class error ID. An unknown tag or a missing validator-option value raises `helper:InvalidValidatorCall`.

`helper.isZero` accepts modes `"num"`, `"add"`, `"vals"`, and `"obj"`. Unknown modes raise `helper:InvalidZeroMode`. A wrong number of mode-specific inputs raises `helper:InvalidZeroCall`. `helper.matSubs` and `helper.bernTbl` deliberately receive a caller-owned error ID, so public indexing and table errors remain in the owning class namespace.

7.2 Class-Private Helpers

Single-class mechanics remain beside their owner rather than in `+helper`. The private area of `pdbase` owns coefficient-tree validation. The private area of `pdmat` owns source fitting, continuity, grid conversion, matrix scanning, and algebraic wrapping. The private area of `pdvar` owns affine and rate-row packing, product routing, value-tree zipping, and symbolic wrapping. The private area of `pdlmi` owns direct and full-box assembly, Gram conversion, and order validation. These functions have no supported direct call form. Their placement is an ownership boundary: a helper should move into a class when only that class can meaningfully use it, and should enter `+helper` only after a real cross-class contract exists.

8

Examples

8.1 Deterministic Full-Box Selection

This example selects the default and an explicit full-box order without solving an optimization problem. The reported values depend only on the public assembly contract, not on incidental YALMIP decision-variable identifiers.

MATLAB input

```
yalmip('clear')
P = pdvar(1, {[0 1]}, Degree=2);
direct = P >= 0;
boxDefault = direct.applyFullBoxPreorder();
boxHigher = boxDefault.applyFullBoxPreorder(2);
polya = direct.applyPolya(1);
boxFromPolya = polya.applyFullBoxPreorder();
Fbox = toYalmip(boxDefault);

fprintf("direct: fullBox=%d, order=%d, constraints=%d\n", ...
    direct.UseFullBoxPreorder, direct.FullBoxOrder, ...
    numel(direct.Constraints));
fprintf("default: fullBox=%d, order=%d, constraints=%d\n", ...
    boxDefault.UseFullBoxPreorder, boxDefault.FullBoxOrder, ...
    numel(boxDefault.Constraints));
fprintf("higher: order=%d, constraints=%d\n", ...
    boxHigher.FullBoxOrder, numel(boxHigher.Constraints));
fprintf("replacement: polya=%d, fullBox=%d, order=%d\n", ...
    boxFromPolya.UsePolya, boxFromPolya.UseFullBoxPreorder, ...
    boxFromPolya.FullBoxOrder);
fprintf("exported constraints=%d\n", length(Fbox));
```


Output

```
direct: fullBox=0, order=0, constraints=3
default: fullBox=1, order=1, constraints=5
higher: order=2, constraints=7
replacement: polya=0, fullBox=1, order=1
exported constraints=5
```

For the degree-two scalar residual, order one uses two PSD Gram blocks followed by three exact coefficient identities. Order two matches a degree-four target and stores two PSD blocks followed by five identities. The original `direct` object remains unchanged, and the selection made from `polya` replaces Pólya rather than elevating its already assembled constraints.

8.2 Parameter-Dependent Lyapunov Solver Smoke

The first solver smoke test in `+tests/+pdlmi/test_solver_smoke.m` contrasts a constant Lyapunov matrix with a degree-one, parameter-dependent one. It solves the same decay and positivity constraints in both cases. The degree-one solution is then materialized as known `pdmatrix` data and plotted. `pdvar` is a decision-expression class and does not itself provide `plot`; applying `value` to its local YALMIP payloads after a successful solve supplies the numeric Bernstein coefficients for `pdmatrix`.

MATLAB input

```
yalmip('clear')
grid = {[0 1]};
A = pdmat(grid, @(x) (1 - x) * [-1 -1; 1 -1] + ...
    x * [-1 -10; 0.1 -1], Degree=1);
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);

Pc = pdvar(2, grid, Degree=0);
CconstantDecay = Pc * A + A' * Pc <= -1e-10 * eye(2);
CconstantPositive = Pc >= eye(2);
solConstant = optimize([CconstantDecay.toYalmip, ...
    CconstantPositive.toYalmip], [], opts);

Pd = pdvar(2, grid);
CdependentDecay = Pd * A + A' * Pd <= -1e-10 * eye(2);
CdependentPositive = Pd >= eye(2);
solDependent = optimize([CdependentDecay.toYalmip, ...
    CdependentPositive.toYalmip], [], opts);
assert(solDependent.problem == 0, solDependent.info)

Pplot = pdmat(grid, ...
    {cellfun(@value, Pd.LocalValues{1}, UniformOutput=false)}, ...
    Degree=Pd.Degree);
figure(Name="Parameter-dependent Lyapunov matrix")
plot(Pplot, SamplesPerCell=40, LineWidth=1.5)
title("Solved parameter-dependent Lyapunov matrix")
```

The figure contains one curve for each entry of the solved 2-by-2 matrix $P(\rho)$, sampled on the parameter interval. It is a diagnostic visualization of the feasible solution, not a certificate beyond the coefficient-wise constraints supplied to the solver. The constant-case result is retained to exercise the comparison in the smoke test; only a MOSEK solve is used there to certify its intended infeasibility distinction.

8.3 Rate-Dependent Block LMI Solver Smoke

This example is adapted from `+tests/+pdlmi/test_solver_smoke.m`. The goal is to find a parameter-dependent $P(\rho)$ and scalar γ such that, on every Bernstein coefficient and rate vertex,

$$\begin{bmatrix} \dot{P} + PA + A^\top P & PB & C^\top \\ B^\top P & -\gamma I & D^\top \\ C & D & -\gamma I \end{bmatrix} \preceq 0, \quad P(\rho) \succeq 0.$$

The derivative term $\dot{P}$ is built with `rhodiff(P, [-1 1])`. The comparison operators create `pdlmi` wrappers, and `toYalmip` hands the coefficient-wise constraints to YALMIP.

MATLAB input

```
yalmip('clear')
grid = linspace(0, 1, 2);
A = pdmat(grid, @(x) [-1, 0.5; -1, -2] + ...
    x * [-1.3, -20; 2, -10], Degree=1);
B = pdmat(grid, @(x) [1, -4; -1, -1] + ...
    x * [2.2, 0.5; -6, -5], Degree=1);
Cmat = eye(2);
Dmat = zeros(2);

P = pdvar(2, grid);
diffP = rhodiff(P, [-1 1]);
gamma = pdvar(1, grid, Degree=0);
E1 = [diffP + P * A + A' * P, P * B, Cmat';
    B' * P, -gamma * eye(2), Dmat';
    Cmat, Dmat, -gamma * eye(2)] <= 0;
E2 = P >= 0;
```

MATLAB input (continued)

```
objective = gamma.LocalValues{1}{1};
solver = 'lmilab';
if exist('mosekopt', 'file') ~= 0
    solver = 'mosek';
end
opts = sdpsettings('solver', solver, 'verbose', 0);
sol = optimize([E1.toYalmip, E2.toYalmip], objective, opts);
assert(sol.problem == 0, sol.info)
gammaValue = value(objective);
assert(isfinite(gammaValue))
fprintf("Optimal H-infinity gamma: %.6g\n", gammaValue)

numel(E1.Constraints)
numel(E2.Constraints)
```

The `fprintf` call prints the actual numeric γ , whose value is solver- and tolerance-dependent; the final two commands report six and two constraints.

A

Bernstein Basis Mathematics

This appendix collects the representation identities used throughout the package. They are exact algebraic identities, not approximation steps. See Farouki’s retrospective for historical and numerical background [1].

A.1 Forward Coordinates And Tensor Basis

On the physical interval $[\rho^-, \rho^+]$, define

$$\alpha = \frac{\rho - \rho^-}{\rho^+ - \rho^-} \in [0, 1], \quad B_i^m(\alpha) = \binom{m}{i} (1 - \alpha)^{m-i} \alpha^i.$$

The endpoint identities are $B_0^m(0) = 1$ and $B_m^m(1) = 1$. Every basis function is nonnegative on the interval and $\sum_i B_i^m = 1$. For $\boldsymbol{\alpha} \in [0, 1]^\ell$, the tensor basis is

$$B_i^m(\boldsymbol{\alpha}) = \prod_{s=1}^{\ell} B_{i_s}^m(\alpha_s).$$

This manual uses the same degree in every coordinate because that is the current storage contract; the identities extend componentwise to mixed degree.

A.2 Exact Monomial Conversion

Let $p(\alpha) = \sum_{j=0}^n a_j \alpha^j = \sum_{k=0}^n b_k B_k^n(\alpha)$. Power coefficients convert to Bernstein coefficients by

$$b_k = \sum_{j=0}^k \frac{\binom{k}{j}}{\binom{n}{j}} a_j, \quad k = 0, \dots, n.$$

The inverse conversion is

$$a_j = \binom{n}{j} \sum_{i=0}^j (-1)^{j-i} \binom{j}{i} b_i, \quad j = 0, \dots, n.$$

For matrices these formulas act entrywise; for tensor polynomials they act along one axis at a time. Basis conversion keeps degree fixed. It is different from degree elevation.

A.3 Lossless Degree Elevation

One elevation step from degree m to $m + 1$ gives

$$\hat{C}_k = \frac{k}{m+1} C_{k-1} + \left(1 - \frac{k}{m+1}\right) C_k,$$

with absent endpoint terms interpreted as zero. Direct elevation from degree m to $M \geq m$ is

$$\hat{C}_k = \sum_{i=\max(0, k-M+m)}^{\min(m, k)} C_i \frac{\binom{m}{i} \binom{M-m}{k-i}}{\binom{M}{k}}, \quad k = 0, \dots, M.$$

Apply the same map successively along each tensor axis. The polynomial is unchanged, and the elevated coefficients are determined convex combinations of the original ones. Elevation therefore adds no decision variables; a new higher-degree `pdvar` parameterization does.

A.4 Products As Weighted Convolution

The basis-product identity

$$B_i^m B_j^n = \frac{\binom{m}{i} \binom{n}{j}}{\binom{m+n}{i+j}} B_{i+j}^{m+n}$$

gives, for ordered matrix factors,

$$C_k = \sum_{i+j=k} A_i B_j \frac{\binom{m}{i} \binom{n}{j}}{\binom{m+n}{k}}.$$

In ℓ dimensions,

$$C_{\mathbf{k}} = \sum_{i+j=\mathbf{k}} A_i B_j \prod_{s=1}^{\ell} \frac{\binom{m}{i_s} \binom{n}{j_s}}{\binom{m+n}{k_s}}.$$

Equivalently, multiply each value grid by its tensor binomial weights, perform ordinary discrete convolution, and divide by the output weights. Matrix factor order must not be exchanged.

A.5 Differentiation

For cell width $h = \rho^+ - \rho^-$,

$$\frac{dp}{d\rho} = \frac{m}{h} \sum_{i=0}^{m-1} (C_{i+1} - C_i) B_i^{m-1}(\alpha).$$

For a tensor polynomial, differentiation with respect to ρ_s takes forward differences along tensor axis s , scales by m/h_s , and leaves the other axes unchanged. Terms in $\dot{P} = \sum_s (\partial P / \partial \rho_s) \dot{\rho}_s$ may then be elevated to a common tensor degree. `rhodiff` evaluates this affine rate expression at the vertices of the supplied rate box and keeps derivative boundary rows cell-local.

A.6 de Casteljau Evaluation And Subdivision

Starting with $C_i^{(0)} = C_i$, de Casteljau recursion is

$$C_i^{(r)} = (1 - \alpha) C_i^{(r-1)} + \alpha C_{i+1}^{(r-1)}, \quad r = 1, \dots, m.$$

The value is $C_0^{(m)}$. The left and right edge chains of the triangular recursion are exactly the Bernstein coefficients on the two subintervals cut at α . Applying subdivision along each tensor axis gives exact coefficients on child boxes. This explains why physical grid refinement can tighten convex-hull evidence while leaving the polynomial unchanged. The current package has no public de Casteljau or adaptive-refinement method.

B

Polynomial Matrix Sign Certificates On A Hypercube

For a residual $F \preceq 0$, write $S = -F$; for $F \succeq 0$, write $S = F$. The certificate problem is $S(\alpha) \succeq 0$ on $[0, 1]^\ell$. Scalar SOS notation below extends to polynomial matrices by using matrix-SOS Gram representations, or equivalently by requiring the scalar polynomial $y^\top S(\alpha)y$ to have the structured certificate. PSD Gram matrices certify non-strict inequalities. Any desired strict margin must first be placed in S .

B.1 Full-Space SOS

The simplest Gram certificate ignores the box and proves global nonnegativity:

$$p(z) = v_d(z)^\top Q v_d(z), \quad Q \succeq 0,$$

where v_d contains monomials through the selected degree. Matching coefficients is linear in Q . This is an SOS certificate on all of $\mathbb{R}^\ell$, so it can be much stronger than positivity only on a box. It is sufficient, but multivariate nonnegative polynomials need not be SOS. The package does not expose a full-space SOS constructor.

B.2 Direct Bernstein Coefficients

If

$$S(\alpha) = \sum_i B_i^m(\alpha) C_i, \quad C_i \succeq 0 \quad \text{for all } i,$$

then $S \succeq 0$ on the box because the basis matrices form a convex combination. This is the package default after residual and rate-vertex assembly. It introduces no Gram matrices. Its converse is false: an indefinite Bernstein coefficient defeats this certificate, not the underlying continuous inequality.

B.3 Pólya-Type Coefficient Relaxation

Pólya's theorem concerns a homogeneous polynomial strictly positive on a simplex: after multiplication by a sufficiently high power of the sum of the simplex coordinates, every coefficient becomes positive. Quantitative bounds are available under strict positivity hypotheses [9]. On each box coordinate, the pair $(1 - \alpha_s, \alpha_s)$ lies on a one-simplex and its sum is one. Multiplication by $\prod_s ((1 - \alpha_s) + \alpha_s)^r = 1$ is therefore exact tensor degree elevation. It can turn a successful strict certificate into visible positive Bernstein coefficients without changing the residual.

`applyPolya(r)` implements one user-selected elevation increment followed by the direct coefficient test. It does not search for a sufficient exponent, add a margin, or add SOS multipliers. Failure at a fixed increment is inconclusive.

B.4 Putinar Quadratic Modules

For a basic closed set $K = \{z : g_j(z) \geq 0, j = 1, \dots, q\}$, the quadratic module generated by the constraints contains polynomials of the form

$$\sigma_0(z) + \sum_{j=1}^q \sigma_j(z)g_j(z), \quad \sigma_j \text{ SOS.}$$

Putinar’s Positivstellensatz states that if this module is Archimedean, then every polynomial strictly positive on K has such a representation [7]. The Archimedean hypothesis is substantive; compactness alone is not the statement of Putinar’s theorem. Degree truncation produces a hierarchy of finite SDPs, and failure at one order is not a nonnegativity counterexample. General Putinar modules are mathematical background here, not a current `pdmi` option.

B.5 Schmüdgen And Full Preorderings

The preordering generated by $g_1, \dots, g_q$ also includes every square-free product:

$$p(z) = \sum_{\nu \in \{0,1\}^q} \sigma_\nu(z) \prod_{j=1}^q g_j(z)^{\nu_j}, \quad \sigma_\nu \text{ SOS.}$$

Schmüdgen’s Positivstellensatz gives this representation for polynomials strictly positive on a compact basic closed semialgebraic set [8]. Unlike Putinar’s result, its standard hypothesis is compactness, but the preordering can require many multiplier blocks.

For the hypercube, the package uses the ℓ generators $h_s = \alpha_s(1 - \alpha_s)$. Their nonnegative set is exactly $[0, 1]^\ell$, and all 2^ℓ subset products form the implemented box-specific full preordering. At a selected absolute order, `applyFullBoxPreorder` creates independent PSD Gram blocks for every physical cell and active rate row, then matches the resulting Bernstein coefficients exactly. It is not a general-domain Schmüdgen interface.

B.6 The One-Dimensional Markov–Lukács Forms

In one variable the interval certificate has exact parity-specific forms. For degree at most $2d$,

$$p(\alpha) = v_d(\alpha)^\top Q_0 v_d(\alpha) + \alpha(1 - \alpha)v_{d-1}(\alpha)^\top Q_1 v_{d-1}(\alpha), \quad Q_0, Q_1 \succeq 0.$$

For degree at most $2d + 1$,

$$p(\alpha) = (1 - \alpha)v_d(\alpha)^\top Q_0 v_d(\alpha) + \alpha v_d(\alpha)^\top Q_1 v_d(\alpha), \quad Q_0, Q_1 \succeq 0.$$

These forms characterize interval nonnegativity at the stated parity, rather than merely supplying one hierarchy level. `pdmi` uses their Bernstein-basis matrix-Gram counterpart when $\ell = 1$ [6].

B.7 Relationships And Implementation Boundary

The methods should not be ordered as if they were interchangeable syntax. Full-space SOS proves a global statement. Direct Bernstein and Pólya coefficient tests exploit convex-hull evidence. Putinar uses a quadratic module and requires an Archimedean hypothesis for its strict-positivity theorem.

Schmüdgen uses a full preordering and compactness. The one-dimensional Markov–Lukács form is exact. Increasing decision degree, elevating a fixed residual, refining the physical grid, and increasing Gram order are four different operations.

The callable package boundary is limited to direct coefficient assembly, fixed-increment Pólya elevation, and fixed-order full box preordering. General full-space SOS, Putinar modules, general-domain preorderings, adaptive order or subdivision, residual diagnostics, and automatic strictness margins are not implemented public APIs. The isolated validation programs under `sos_validation/` compare formulations; they are not runtime dependencies and do not expand the package interface.

References

- [1] R. T. Farouki, “The Bernstein polynomial basis: A centennial retrospective,” *Computer Aided Geometric Design*, vol. 29, no. 6, pp. 379–419, 2012. [doi:10.1016/j.cagd.2012.03.001](https://doi.org/10.1016/j.cagd.2012.03.001).
- [2] M. Zettler and J. Garloff, “Robustness analysis of polynomials with polynomial parameter dependency using Bernstein expansion,” *IEEE Transactions on Automatic Control*, vol. 43, no. 3, pp. 425–431, 1998. [doi:10.1109/9.661615](https://doi.org/10.1109/9.661615).
- [3] I. Masubuchi, A. Kume, and E. Shimemura, “Spline-type solution to parameter-dependent LMIs,” in *Proceedings of the 37th IEEE Conference on Decision and Control*, vol. 2, 1998. [doi:10.1109/CDC.1998.758549](https://doi.org/10.1109/CDC.1998.758549).
- [4] Y. Xu and F. Jabbari, “Spline-based parameter varying output feedback synthesis with improved L_2 gain,” in *2024 IEEE 63rd Conference on Decision and Control*, pp. 1455–1460, 2024. [doi:10.1109/CDC56724.2024.10886461](https://doi.org/10.1109/CDC56724.2024.10886461).
- [5] G. Chesi, “LMI techniques for optimization over polynomials in control: a survey,” *IEEE Transactions on Automatic Control*, vol. 55, no. 11, pp. 2500–2510, 2010. [doi:10.1109/TAC.2010.2046926](https://doi.org/10.1109/TAC.2010.2046926).
- [6] F. Lukács, “Verschärfung des ersten Mittelwertsatzes der Integralrechnung für rationale Polynome,” *Mathematische Zeitschrift*, vol. 2, pp. 295–305, 1918. [EuDML record](https://www.eudml.org/doc/111111).
- [7] M. Putinar, “Positive polynomials on compact semi-algebraic sets,” *Indiana University Mathematics Journal*, vol. 42, no. 3, pp. 969–984, 1993. [JSTOR 24897130](https://www.jstor.org/stable/24897130).
- [8] K. Schmüdgen, “The K -moment problem for compact semi-algebraic sets,” *Mathematische Annalen*, vol. 289, pp. 203–206, 1991. [doi:10.1007/BF01446568](https://doi.org/10.1007/BF01446568).
- [9] V. Powers and B. Reznick, “A new bound for Pólya’s theorem with applications to polynomials positive on polyhedra,” *Journal of Pure and Applied Algebra*, vol. 164, nos. 1–2, pp. 221–229, 2001. [doi:10.1016/S0022-4049\(00\)00155-9](https://doi.org/10.1016/S0022-4049(00)00155-9).